

SlyLED User Manual

CONTENTS

- [SlyLED User Manual — 3D Volumetric Lighting System \(v1.0\)](#)
 - [Table of Contents](#)
 - [1. Getting Started](#)
 - [Quick Start](#)
 - [2. Walkthrough: First Show in 30 Minutes](#)
 - [Step 1 — Launch and Create a New Project](#)
 - [Step 2 — Set Stage Dimensions](#)
 - [Step 3a — Discover DMX Hardware](#)
 - [Step 3b — Configure and Start the DMX Engine](#)
 - [Step 4 — Add DMX Moving Head Fixtures](#)
 - [Step 5 — Add a Wash or Spot Fixture](#)
 - [Step 6 — Register Camera Nodes](#)
 - [Step 7 — Position All Fixtures on the Layout](#)
 - [Step 8 — Add a Stage Object](#)
 - [Step 9 — Run Mover Calibration](#)
 - [Step 10 — Create Actions](#)
 - [Step 11 — Build a Timeline](#)
 - [Step 12 — Bake and Start Playback](#)
 - [Step 13 — Save the Project](#)
 - [Walkthrough Troubleshooting](#)
 - [3. Platform Guide](#)
 - [Windows Desktop \(SPA\)](#)
 - [Android App](#)
 - [Firmware Config \(ESP32/D1 Mini\)](#)
 - [4. Fixture Setup](#)
 - [What Are Fixtures?](#)
 - [Adding LED Fixtures](#)

- Adding DMX Fixtures (Wizard)
- DMX Monitor
- Fixture Group Control
- Testing DMX Channels
- Fixture Types
- 5. Stage Layout
 - 2D Canvas
 - 3D Viewport (Desktop Only)
 - Move / Rotate Mode
 - Coordinate System
- 6. Stage Objects
 - Object Types
 - Stage-Locked Objects
 - Mobility
 - Patrol Motion
 - Temporal Objects
- 7. Creating Spatial Effects
 - Spatial Effects vs Classic Actions
 - Creating a Spatial Effect
- 8. Track Action
 - How a Track action runs
 - Assignment algorithm
 - Action editor — Advanced expander (#811)
 - Target object selection
 - Cooperation with shows and remotes (#763 / #835)
- 9. Building a Timeline
- 10. Baking & Playback
 - Bake
 - Output
- 11. Show Preview Emulator

- [Dashboard Preview](#)
- [Desktop SPA](#)
- [Android App](#)
- [Spatial Field Visualization](#)
- [DMX-Only Rigs](#)
- [12. DMX Fixture Profiles](#)
 - [Built-in Profiles](#)
 - [Profile Editor — Step-by-Step \(#527\)](#)
 - [Global brightness scaling \(#843\)](#)
 - [Aim cone clamping \(#803\)](#)
 - [Legacy quick-reference](#)
 - [Browsing the Open Fixture Library](#)
 - [Community Fixture Library](#)
 - [Import/Export](#)
- [13. Preset Shows](#)
 - [Available themes](#)
 - [Track-action vs sweep](#)
 - [Customising an installed preset](#)
 - [Behaviour outside playback](#)
- [14. Camera Nodes](#)
 - [Adding a Camera Node](#)
 - [Camera Config Page](#)
 - [Snapshots](#)
 - [Object Detection](#)
 - [Camera Deploy](#)
 - [Multi-Camera Support](#)
 - [Environment Scanning](#)
 - [Per-Camera Fixtures](#)
 - [Tracking Configuration](#)
 - [Mover Calibration](#)
 - [Fixture Orientation Test](#)
- [15. Firmware & OTA Updates](#)

- Current production versions (orchestrator v1.7.83)
- USB Flash
- OTA (Over-the-Air)
- Firmware Registry
- 16. System Limits
- 17. Troubleshooting
- 18. Examples
 - Example A: Camera Tracking — Moving Heads Follow a Person (#376)
 - Example B: Mover Tracking with Spatial Effects (#379)
 - Example C: Manual Mover Calibration (#381)
 - Example D: Camera Calibration with ArUco Markers (#380)
 - Example E: Spotlight Follow Person — Live Tracking Preset (#382)
- 19. API Quick Reference
 - Stage & Layout
 - Fixtures
 - Shows & Timelines
 - DMX
 - Cameras
 - Camera Node Local API (port 5000)
- 20. Glossary
- Appendix A — Camera Calibration Pipeline (DRAFT)
 - A.1 Pipeline overview
 - A.2 ArUco marker surveying
 - A.3 Intrinsic calibration
 - A.4 Extrinsic solve (solvePnP)
 - A.5 Dark-reference capture (#651)
 - A.6 Point cloud + multi-camera merge
 - A.7 Surface analysis (RANSAC)
 - A.8 Beam detection (interface used by mover calibration)
 - A.9 Rotation schema v2 (axis convention)
 - A.10 Failure modes & operator expectations
 - A.11 File locations & persistence

- [Appendix B — Moving-Head Calibration Pipeline](#)
 - [B.1 Pipeline overview](#)
 - [B.2 Phase reference](#)
 - [B.3 Phase-by-phase detail](#)
 - [B.4 Time budget + blackout-on-timeout \(#653\)](#)
 - [B.5 Abort path](#)
 - [B.6 Failure modes & operator diagnostics](#)
 - [B.7 Tuning-parameter reference](#)
 - [B.8 Related features](#)
- [Appendix C — Documentation Maintenance](#)
 - [C.1 Source-of-truth files](#)
 - [C.2 Reviewer checklist \(short form\)](#)
 - [C.3 Regenerating the manual](#)
 - [C.4 Enforcement](#)
 - [C.5 DRAFT banner removal](#)
- [Appendix D — Idle Behaviour](#)
 - [What “parked” means](#)
 - [When the orchestrator parks a head](#)
 - [What does NOT trigger a park](#)
 - [How to verify on your rig](#)
- [Appendix E — Remote Control: Android Phone & Gyro Puck](#)
 - [Claim lifecycle](#)
 - [Gestures](#)
 - [Claim arbitration with shows \(#763\)](#)
 - [Colour & dimmer during a claim \(#814\)](#)
 - [Recovery from divergent state](#)

SlyLED User Manual — 3D Volumetric Lighting System (v1.0)

Licensing. SlyLED is distributed under the [PolyForm Noncommercial 1.0.0](https://electricrv.ca/slyled/licensing) licence — **free for personal, hobbyist, educational, research, and nonprofit use**. Paid client engagements, corporate internal use, embedded products, and paid services built on SlyLED require a separate commercial licence. See <https://electricrv.ca/slyled/licensing> or write to info@electricrv.ca.

Table of Contents

1. [Getting Started](#)
2. [Walkthrough: First Show in 30 Minutes](#)
3. [Platform Guide](#)
4. [Fixture Setup](#)
5. [Stage Layout](#)
6. [Stage Objects](#)
7. [Creating Spatial Effects](#)
8. [Track Action](#)
9. [Building a Timeline](#)
10. [Baking & Playback](#)
11. [Show Preview Emulator](#)
12. [DMX Fixture Profiles](#)
13. [Preset Shows](#)
14. [Camera Nodes](#)
15. [Firmware & OTA Updates](#)
16. [System Limits](#)
17. [Troubleshooting](#)
18. [Examples](#)
19. [API Quick Reference](#)
20. [Glossary](#)
21. [Appendix A — Camera Calibration Pipeline](#)
22. [Appendix B — Moving-Head Calibration Pipeline](#)
23. [Appendix C — Documentation Maintenance](#)
24. [Appendix D — Idle Behaviour](#)
25. [Appendix E — Remote Control: Android Phone & Gyro Puck](#)

1. Getting Started

SlyLED is a three-tier LED and DMX lighting control system: - **Orchestrator** (Windows/Mac desktop app or Android app) — design shows and control playback - **Performers** (ESP32/D1 Mini) — run LED effects on hardware - **DMX Bridge** (Giga R1 WiFi) — output Art-Net/sACN to DMX fixtures

Quick Start

1. Launch the desktop app: `powershell -File desktop\windows\run.ps1` (Windows) or `bash desktop/mac/run.sh` (Mac)
 2. Open the browser at `http://localhost:8080`
 3. Go to **Setup** tab, click **Discover** to find performers on your network
 4. Go to **Layout** tab to position fixtures on the stage
 5. Go to **Runtime** tab, load a **Preset Show**, click **Bake & Start**
-

2. Walkthrough: First Show in 30 Minutes

This walkthrough builds a complete DMX moving-head show from scratch — hardware discovery, fixture setup, layout, camera registration, actions, timeline, and playback. Every step was validated end-to-end during QA testing (issue #533). Follow in order; each step builds on the last.

What you need: - SlyLED orchestrator running on Windows or Mac - At least one DMX moving head connected via Art-Net/sACN bridge (e.g. Enttec ODE Mk3) - At least one USB camera node on the network (Orange Pi or Raspberry Pi) - All devices on the same LAN subnet as the orchestrator

Step 1 — Launch and Create a New Project

Start the orchestrator:

```
powershell -File desktop\windows\run.ps1
```

Open `http://localhost:8080` in Chrome or Edge. The SPA loads on the Dashboard tab.



SPA at launch showing Dashboard tab

SPA at launch showing Dashboard tab

Go to **Settings** tab → **Project** → click **New Project**, then name it (e.g. “Walkthrough Show”).



New project dialog

New project dialog

Step 2 — Set Stage Dimensions

In **Settings** → **Stage**, enter the dimensions of your performance area: - Width: 6000 mm (6 m) - Height: 3000 mm (3 m) - Depth: 4000 mm (4 m)

Click **Save**. The layout canvas will resize to match these dimensions.

Step 3a — Discover DMX Hardware

Go to the **Setup** tab. In the **DMX Nodes** section, click **Discover Nodes**. SlyLED broadcasts an ArtPoll packet; Art-Net bridges on the network reply within 3 seconds.



Setup tab after hardware discovery — Art-Net node shown

Setup tab after hardware discovery — Art-Net node shown

Any discovered nodes appear in the list with their IP, port, and universe count. If your bridge is not found: - Confirm it is powered and on the same LAN subnet - Check that UDP port 6454 is not blocked by a local firewall - Some bridges require the Art-Net source IP to match their configured subnet

Step 3b — Configure and Start the DMX Engine

Go to **Settings** → **DMX**:

1. **Universe Routing**: Set Universe 1 → your Art-Net node IP (or leave as broadcast `255.255.255.255` to reach all nodes on the subnet).
2. Click **Start Engine**. The status indicator turns green (“Running”).

 DMX engine configuration — universe routing and start

 DMX routing — universe 1 assigned to bridge

Important: The engine must be running before adding DMX fixtures or running calibration. If you stop and restart the orchestrator, re-start the engine here.

Step 4 — Add DMX Moving Head Fixtures

Go to the **Setup** tab → click + **DMX Fixture**. The fixture wizard opens.

Finding the right profile: 1. In the **Search** box, type your fixture’s name (e.g. “Sly Moving Head Super Mini”) 2. Results show Local profiles first, then Community (from the shared library), then OFL (Open Fixture Library) 3. If a community profile download fails (“imported: 0”), it may contain unsupported channel types — fall back to a local generic profile or search OFL directly 4. For a generic 16-channel moving head with no exact match, search OFL for “moving head” and import the closest match

Fixture 1 (stage left): - Name: `MH1 SL` - Universe: 1, Start Address: 1 - Profile: your moving head profile - Click **Create Fixture**



Moving head 1 added to Setup tab

Moving head 1 added to Setup tab

Fixture 2 (stage right): - Name: `MH2 SR` - Universe: 1, Start Address: 17 - Profile: same profile - Click **Create Fixture**



Moving head 2 added to Setup tab

Moving head 2 added to Setup tab

Step 5 — Add a Wash or Spot Fixture

Add any additional fixtures (e.g. a 350W wash spot): - Name: - Universe: 1, Start Address: 33 - Profile: your spot/wash profile



350W spot added to Setup tab

350W spot added to Setup tab

Step 6 — Register Camera Nodes

Still on the **Setup** tab, scroll to the **Camera Nodes** section. Click **Discover Cameras**. Camera nodes running respond to the same UDP broadcast as performers.

Alternatively, enter the camera's IP manually and click **Add**.

Camera 1 (left): - IP: your camera's IP (e.g.) - Name:

Camera 2 (right): - IP: your second camera's IP - Name:



Two cameras added to Setup tab

Two cameras added to Setup tab

Each camera appears with online/offline status. Click **Snap** to verify the live feed.



Camera 1 snapshot — left-side view



Camera 2 snapshot — right-side view

Note: Camera discover sometimes returns 0 nodes on the first broadcast due to UDP timing. If no cameras are found, wait 3 seconds and click **Discover** again. This is a known intermittency (#542) being addressed in a future release.

Step 7 — Position All Fixtures on the Layout

Switch to the **Layout** tab. All added fixtures appear in the left sidebar as “unplaced.”

Place and position each fixture:

1. Click a fixture in the sidebar to select it
2. Click on the canvas to place it, or drag from the sidebar
3. Double-click the placed fixture to open the edit dialog and enter exact coordinates

FIXTURE	X (MM)	Y (MM)	Z (MM)
MH1 SL	1500	3000	500
MH2 SR	4500	3000	500
Spot C	3000	3000	500
Cam Left	0	2500	0
Cam Right	6000	2500	0

Click **Save** after entering coordinates for each fixture.

 Layout tab — fixtures placed at initial positions

 Layout tab — all fixtures positioned

 Camera fixtures positioned on layout

Tip: Use 3D view (toggle in the layout toolbar) to visually verify that movers are elevated on the truss and aimed downward toward the stage floor.

Step 8 — Add a Stage Object

Go to the **Layout** tab → click + **Object** in the toolbar.

- **Name:**
- **Type:** Prop (moving — trackable by movers)
- **Position:** X: 3000, Y: 0, Z: 2000 (center stage, at floor level, mid-depth)
- **Size:** 300 × 1200 × 300 mm

Click **Save**. The object appears as a labeled rectangle on the canvas.



Music stand object on layout

Music stand object on layout

Step 9 — Run Mover Calibration

Before the moving heads can accurately track positions, calibrate each one. This step requires the camera nodes to be positioned on the layout (Step 7) and the DMX engine running (Step 3b).

In the **Layout** tab, double-click MH1 SL. Click **Calibrate**.



Calibration buttons in fixture edit dialog



Calibration wizard UI

- Select **Green** as the beam color (good contrast on dark floors)
- Click **Start Calibration**
- The wizard runs automatically through eight phases: warmup → discovery → blink-confirm → mapping/convergence → grid build → verification sweep → model fit → held-out parametric gate → save
- Repeat for MH2 SR

Calibration typically takes 2–4 minutes per head. For the complete phase-by-phase reference — what each phase does, how long it should take, what fallbacks exist, and what to check when a phase stalls — see [Appendix B — Moving-Head Calibration Pipeline](#).

Step 10 — Create Actions

Go to the **Actions** tab. You'll create two actions: a static aim and a figure-eight sweep.

Action 1: Aim Red (static spotlight) 1. Click + **New Action** 2. **Name:** Aim Red 3. **Type:** DMX Scene 4. **Colour:** Red (255, 0, 0) 5. **Dimmer:** 255 6. Click **Save Action**



Aim Red action — aimed at stage center

Aim Red action — aimed at stage center

Action 2: Figure Eight (dynamic sweep) 1. Click + **New Action** 2. **Name:** 3. **Type:** 4. **Target Objects:** leave empty (track all moving objects) 5. **Cycle Time:** 4000 ms 6. Click **Save Action**



Figure Eight track action

Figure Eight track action

Step 11 — Build a Timeline

Go to the **Runtime** tab (labeled **Shows** in some versions). Click + **New Timeline**.

A dialog prompts for the name — enter . A second dialog prompts for duration — enter (seconds). Click OK.



Timeline editor with tracks

Timeline editor with tracks

Add tracks:

For each fixture or group, click + **Add Track**: - Track for — add clip: at 0s, duration 10s - Track for — add clip: at 10s, duration 110s - Track for — add clip: at 0s, duration 120s - Track for — add clip with ambient wash color

The Track action (type 18) evaluates in real-time during playback and doesn't need to be baked per-frame — it reads live object positions at 40 Hz.

Step 12 — Bake and Start Playback

1. Click **Bake** — the engine compiles the timeline into per-fixture action sequences. Progress shows frame count.

2. Click **Start** — NTP-synchronized playback begins.

Watch the **Runtime** view: - Beam cones animate in 3D as the timeline plays - The figure-eight pattern moves through the stage space - DMX output is sent via Art-Net to the physical fixtures



Runtime view with animated beam cones

Runtime view with animated beam cones

To test a blackout: - Click **Stop**, then fire a **Blackout** action from the Settings → Group Control panel



Blackout state — all beams off

Blackout state — all beams off

Step 13 — Save the Project

Go to **Settings** → **Project** → click **Export**. A `.slyshow` file is downloaded containing all fixtures, layout positions, objects, camera registrations, calibration data, actions, and timelines.

To reload: Settings → Project → **Import** → select the `.slyshow` file.



Project saved — all state bundled in .slyshow file

Project saved — all state bundled in .slyshow file

Walkthrough Troubleshooting

PROBLEM	SOLUTION
No Art-Net nodes discovered	Confirm bridge is on the same subnet; UDP port 6454 not blocked
DMX engine won't start	Check Settings → DMX → verify universe routing is configured
Community profile download fails	Profile has unsupported channel types — use local or OFL profile instead
Fixture position resets to 0,0,0	Ensure <code>saveFixture()</code> completes before switching tabs; use the edit dialog Save button
Camera discover returns 0	Wait 3s and retry — first broadcast may arrive before socket is bound (#542)
Calibration fails to detect beam	Dim ambient light, verify beam color contrasts with floor, check camera can see the beam
Figure Eight doesn't move heads	Verify Track action has no <code>trackFixtureIds</code> restriction; confirm engine is running
Timeline tracks missing after create	Add tracks manually after timeline creation — they are not auto-created

3. Platform Guide

Windows Desktop (SPA)

The primary design and control interface. Full-featured 7-tab SPA with 2D/3D layout, timeline editor, spatial effects, DMX profiles, and firmware management.

Launch: `powershell -File desktop\windows\run.ps1` or run `SlyLED.exe` **Install:** Run `SlyLED-Setup.exe` (includes system tray icon)

Android App

Live operator tool for running shows from your phone. Connects to the desktop server over WiFi.

Install: Transfer `SlyLED-debug.apk` to your phone and install. **Connect:** Scan the QR code on the desktop Settings tab, or enter the server IP and port manually.



Android Stage View

Android Stage View

Android Screens: - **Stage** — live viewport showing all fixtures (LED, DMX, cameras) with beam cones, tracked object markers, and grid floor. Pinch-to-zoom and drag-to-pan. HUD shows current show status. Play/Stop button and brightness slider at bottom. - **Control** — one-tap timeline start, playlist with loop toggle, global brightness slider, and Pointer Mode for aiming DMX moving heads with your phone's gyroscope. - **Status** — device monitoring (performers online/offline, RSSI, firmware), camera nodes with Track button to start/stop person tracking, and Art-Net/DMX engine status. - **Settings** — server name, stage dimensions, dark mode, config export/import, disconnect.



Android Control



Android Status

Pointer Mode: Select a DMX moving head on the Control tab and tap its name under Pointer Mode. Hold your phone and point where you want the light — the fixture's pan/tilt follows your phone orientation in real-time at 20 Hz. Tap Recenter to calibrate, X to exit.

Firmware Config (ESP32/D1 Mini)

Each performer serves a 3-tab config page at `http://<device-ip>/config`: - **Dashboard** — hostname, firmware version, active action status - **Settings** — device name, description, string count - **Config** — per-string LED count, length, direction, GPIO pin (ESP32)

4. Fixture Setup

What Are Fixtures?

A fixture is the primary entity on the stage. It wraps physical hardware and adds stage-level attributes: - **LED fixtures** — linked to a performer child, with LED strings - **DMX fixtures** — linked to a DMX universe/address, with a profile and aim point

Adding LED Fixtures

1. Go to **Setup** tab, click **Discover** to find performers
2. Click **Add Fixture** → select “LED” type
3. Link to a performer and configure strings (LED count, length, direction)

Adding DMX Fixtures (Wizard)

Click + **DMX Fixture** on the Setup tab to launch the 3-step wizard: 1. **Choose Fixture:** Search the Open Fixture Library (700+ fixtures) or create a custom fixture 2. **Set Address:** Universe, start address, and name — with real-time conflict detection 3. **Confirm:** Review all settings, click “Create Fixture”

DMX Monitor

Settings → DMX → **DMX Monitor** opens a real-time 512-channel grid per universe. Click any cell to set a value. Color-coded by intensity.

Fixture Group Control

Settings → DMX → **Group Control** opens a control panel for fixture groups. Master dimmer slider, R/G/B sliders, and quick color preset buttons (Warm, Cool, Red, Off).

Testing DMX Channels

On the Setup tab, click **Details** on any DMX fixture to open the channel test panel: - **Sliders** for every channel with live DMX output - **Quick buttons:** All On, Blackout, White, Red, Green, Blue - **Capability labels** show what each value range does (e.g., “Strobe slow→fast”) - Changes take effect immediately on the physical fixture via Art-Net/sACN

Fixture Types

TYPE	DESCRIPTION
Linear	LED strip. Pixels along a path.
Point	DMX light source with beam cone.
Group	Collection of fixtures targeted as one.

5. Stage Layout

2D Canvas

The Layout tab shows a 2D front view of the stage. Stage dimensions (width × height) are set in Settings.

The layout toolbar provides: Save, 2D/3D mode toggle (shows current mode as text), Recenter, Top view, Front view, Auto-arrange DMX, Show/hide LED strings. Active toggles highlight in green.

Use the `?tab=` URL parameter for deep-linking directly to any tab (e.g., `?tab=layout`).

ACTION	DESKTOP	ANDROID
Place fixture	Drag from sidebar	Tap fixture then tap canvas
Move fixture	Drag on canvas	Drag on canvas
Remove fixture	Double-click → Remove	Tap → Edit → Remove
Zoom	Scroll wheel	Pinch gesture
Pan	—	Two-finger drag
Edit coordinates	Double-click	Tap placed fixture
Edit object	Double-click	Tap object in list

What's rendered: - Grid lines at 1-meter spacing - Stage dimension labels - **LED fixtures:** Green nodes with colored string lines (direction arrows) - **DMX fixtures:** Purple nodes with beam cone triangles toward aim point - **Objects:** Semi-transparent rectangles with name labels (clipped to stage) - **Aim dots:** Red circles at DMX aim points

3D Viewport (Desktop Only)

Toggle to 3D mode for an interactive Three.js scene: - Orbit camera with mouse drag - Beam cones as 3D geometry - Draggable aim spheres for DMX fixtures - Object planes/boxes with transparency

Move / Rotate Mode

The layout canvas has two interaction modes, toggled with keyboard shortcuts or the toolbar button (which displays **M** or **R** to show the active mode):

KEY	MODE	DESCRIPTION
M	Move	Drag any placed fixture to reposition it (default mode)
R	Rotate	Click a DMX fixture or camera fixture to show a compass ring; drag around the ring to aim the fixture

Rotate mode details: - A compass ring appears around the selected fixture when you enter Rotate mode - Drag clockwise or counter-clockwise to set the horizontal aim direction - The beam cone updates in real-time as you drag - In 3D viewport, Rotate mode activates Three.js **TransformControls** in rotate mode — drag the colored arcs to rotate on any axis - Press **Ctrl+Z** to undo the last move or rotation

Typical workflow: Place all fixtures in Move mode, then switch to Rotate mode to aim DMX moving heads and cameras toward their intended focus areas before running calibration.

Coordinate System

- **aimPoint[0]** = X (horizontal position, mm)
- **aimPoint[1]** = Y (height from floor, mm) — used for 2D canvas vertical axis
- **aimPoint[2]** = Z (depth, mm) — used in 3D viewport only
- **canvasW** = stage width × 1000 (mm)
- **canvasH** = stage height × 1000 (mm)

6. Stage Objects

Objects represent physical elements on the stage — walls, floors, trusses, screens, and props or performers — plus the abstract “targets” that Track actions chase. Anything a moving head needs to aim at lives in the Objects tab.

The tab was previously called “Surfaces”; the rename to **Objects** shipped with v1.7.30. Old project files import cleanly.

Object Types

TYPE	DEFAULT MOBILITY	DESCRIPTION
Wall	Static	Back wall, stage-locked to stage width × height
Floor	Static	Stage floor, stage-locked to stage width × (depth + 1 m)
Truss	Static	Lighting truss bar
Screen	Static	Projection surface
Prop	Moving	Performer, set piece, or mobile element
Custom	Moving	User-defined object
Ribbon target	Moving	Travelling stage-coord anchor used by the Aurora Curtain template (#839) — a coordinated rig moves through the same point along a chosen axis

Stage-Locked Objects

Wall and floor objects can be locked to stage dimensions. When you change the stage size in Settings → Stage, locked objects resize automatically.

Mobility

- **Static** — fixed position. Cannot be tracked by moving heads.
- **Moving** — position can change at runtime. Trackable by DMX moving heads via the Track action (chapter 8).

Patrol Motion

Moving objects can patrol during playback. Each patrol carries a **pattern**, an axis or shape, a cycle time, and an optional easing curve. The patrol evaluator runs at 40 Hz inside the DMX playback loop, immediately before Track actions read object positions, so a patrolling target's pose is always one frame fresh by the time a mover reads it.

Patterns

PATTERN	GEOMETRY	USE IT FOR
Ping-pong	Travels from one bounding-box corner to the opposite, then reverses	A side-stage performer's predicted path
Circle	Constant-radius loop around the bounding-box centre	A turntable performer or a rotating stage gag
Figure-8	Lemniscate (two-lobe) inside the bounding box	A complex path that visits two foci — useful for cross-stage chases
Square	Four straight-line legs around the bounding-box rim	A “patrol perimeter” feel used in security / industrial themes
Ribbon <i>(new in v1.7.83)</i>	A single travelling anchor on a chosen axis (left-right, front-back, up-down, cross, figure-8); multiple movers ride the ribbon at phase offsets so a sweep visibly travels the rig instead of every head moving in unison	The Aurora Curtain template's coordinated curtain effect

Speed

- **Slow** — 20 s cycle.
- **Medium** — 10 s cycle.
- **Fast** — 5 s cycle.
- **Custom** — set `cycleS` directly. The default `speedPreset` is `medium`; ribbon patrol objects ship with `speedPreset: "custom"` so the template's `cycleS: 12` wins instead of the medium default.

Range

Bounding-box percentage (default 10 %–90 %). The patrol stays inside the box; targets near a wall don't lead a mover into IK gimbal lock.

Easing

- **Sine** — smooth acceleration / deceleration (default).
- **Linear** — constant velocity.

patrolMode

Patrol objects can be set to one of:

- `always` (*default*) — patrol motion runs whenever the orchestrator is up. Useful for “the prop is alive even before the show starts”.
- `on-demand` — patrol motion is suspended until an active timeline references this object via a Track action. The instant a referencing clip starts playing, the patrol resumes from where it would have been if it had been running all along (so a Track action that expects a moving target sees one immediately, not a stationary prop). Operators are sometimes surprised to find an `on-demand` patrol prop sitting still on the stage view — that’s the intended behaviour.

Temporal Objects

External systems can create short-lived objects via `POST /api/objects/temporal`:

- Always in-memory; never saved to disk.
- Require `ttl > 0` (time-to-live in seconds).
- Auto-expire when TTL elapses.
- Position updates refresh the TTL.
- Shown in the runtime viewer with a dashed outline and a countdown badge.
- The camera tracker pushes detected people in via this route — each detection becomes a temporal object that the Spotlight Follow Person preset’s Track action chases.

The temporal-object scale uses the renderer’s `[width, height (Z), depth (Y)]` ordering — call sites that want a “person-shaped” detection should ship `[0.6, 1.8, 0.6]` (60 cm wide, 1.8 m tall, 60 cm deep), not `[0.6, 0.6, 1.8]`.

7. Creating Spatial Effects

Spatial Effects vs Classic Actions

- **Classic Actions** (Solid, Chase, Rainbow, etc.): Run locally on each performer. Pattern based on pixel index. When assigned to DMX fixtures, classic actions are automatically converted to DMX Scene segments with appropriate dimmer, pan/tilt defaults.
- **DMX Actions**: Control DMX-specific features directly:

- **DMX Scene** — Set exact values for dimmer, pan, tilt, strobe, gobo, color wheel, prism
- **Pan/Tilt Move** — Animate pan/tilt from start to end position over time
- **Gobo Select** — Select a gobo wheel position
- **Color Wheel** — Select a color wheel position
- **Track** (Type 18) — Make moving heads follow moving objects in real-time (see [Track Action](#))
- **Spatial Effects:** Operate in 3D space. A sphere of light sweeping across the stage illuminates different fixtures at different times.

SlyLED supports 19 action types in total: 14 classic LED actions plus 5 DMX/spatial actions (DMX Scene, Pan/Tilt Move, Gobo Select, Color Wheel, Track).

Creating a Spatial Effect

Navigate to **Actions** tab → + **New Spatial Effect**.

FIELD	DESCRIPTION
Shape	Sphere, Plane, or Box
Color	RGB color applied to pixels inside the field
Size	Radius (sphere), thickness (plane), or dimensions (box)
Motion Start/End	3D positions in millimeters
Duration	Travel time from start to end
Easing	Linear, ease-in, ease-out, ease-in-out
Blend	Replace, Add, Multiply, Screen

8. Track Action

The Track action (type 18) is the bridge between the Objects tab (chapter 6) and the moving-head rig: while a clip with a Track action is playing, every assigned moving head computes its pan / tilt aim from the live position of one or more target objects, every frame (40 Hz). When the clip ends — or the action's last referencing clip finishes — the assigned heads park at their Home pose (#807).

How a Track action runs

1. Place the moving objects you want chased on the **Layout / Objects** tab (chapter 6) — props, custom moving objects, ribbon targets, or camera-detected people coming in as temporal objects.
2. Create a Track action on the **Actions** tab. Set its name, scope, and target list.
3. Drop a clip on a track in the **Timeline** that references the Track action.
4. During playback, the 40 Hz DMX loop reads each target’s current position, picks (or assigns) one or more heads to chase it, runs the canonical aim-vector IK (#806 / #809), and writes pan / tilt to the universe buffer. The 3D viz’s beam cone is driven from the same aim vector, so the visualisation always matches what the physical head is doing.

Assignment algorithm

The assignment is recomputed every frame from `trackFixtureIds` (the heads in scope) and the resolved target list:

SITUATION	BEHAVIOUR
Equal number of heads and objects	1 : 1 mapping by index
More heads than objects, trackAutoSpread = false	Each object is chased by exactly one head; extra heads sit idle (still on, just not assigned).
More heads than objects, trackAutoSpread = true	Spread the heads across each object’s width — useful for washing a single target with multiple movers.
More objects than heads, default	Cycle through objects with one head landing on a different target every <code>trackCycleMs</code> (default 2000 ms).
More objects than heads, trackFixedAssignment = true	Each head locks to one target by index; extras are ignored until a head frees up.

Per-fixture offsets (`trackFixtureOffsets`) allow asymmetric rigs to aim each head at a slightly different point on the same target — for example, a stage-left mover aims at the performer’s head while a stage-right mover aims at the feet.

Action editor — Advanced expander (#811)

The Action editor’s DMX Scene-family panes group the simple fields (name, scope, RGB, dimmer) at the top, with an **Advanced** expander underneath holding the per-action fine controls. For Track actions the Advanced section exposes:

FIELD	WHAT IT CONTROLS	RANGE
Cycle Time (ms)	<code>trackCycleMs</code> — only used when there are more objects than heads	100 – 10 000 ms (clamped)
Offset X / Y / Z (mm)	Global aim offset added to every target. Useful for “aim 30 cm above the floor where the marker actually is” or “aim at head height instead of centre of mass”.	± 10 000 mm
Auto-spread	Toggles <code>trackAutoSpread</code> for the more-heads-than-objects case	—
Fixed assignment	Toggles <code>trackFixedAssignment</code> — disables cycling, locks each head to one target	—
Track dimmer	<code>trackDimmer</code> — overrides the dimmer the heads run at while the Track action is active. Defaults to 255 (full); operators sometimes lower this so the followed beam doesn’t overpower the rest of the rig.	0 – 255

Target object selection

FIELD	WHAT IT DOES
<code>trackObjectIds</code>	An explicit list of object IDs to track. Wins over <code>trackObjectType</code> when set. Searches the whole moving-object set (including patrol props), so an <code>on-demand</code> patrol prop’s id resolves cleanly here.
<code>trackObjectType</code>	Filter moving objects by type — e.g. <code>"prop"</code> , <code>"ribbon-target"</code> , <code>"custom"</code> . Useful when the target set is “every ribbon target” without listing IDs.
<code>trackMode</code>	Only consulted when <code>trackObjectIds</code> and <code>trackObjectType</code> are both empty. <code>"camera-moving"</code> (<i>default</i>) — track only camera-detected people (temporal objects). <code>"all-moving"</code> — track patrol props and camera detections together.

If a Track action has explicit `trackObjectIds` but every listed object has been deleted, the action skips the frame entirely rather than blacking out the assigned heads — pre-fix (v1.7.78 and earlier) the missing-target case would zero the dimmer of every head in scope.

Cooperation with shows and remotes (#763 / #835)

Track actions are full citizens of the show timeline:

- An orphan Track action (one that lives in `_actions` but is not referenced by any clip in the running timeline) does **not** evaluate (#835). This stops a leftover preset action from blacking out movers in unrelated timelines.
- When a remote (Android phone or gyro puck) claims a head via the mover-control claim arbiter (chapter on Remote control), the claimed head is muted from the Track action for the duration of the claim — operator gestures take priority over the show. Releasing the claim returns the head to the show without a re-bake.
- Track actions write the canonical aim-vector through the same `_set_canonical_aim_stage` path the rest of the orchestrator uses, so calibrate-end during a Track-driven show observes the head's true direction without an inverse-IK round-trip.

9. Building a Timeline

1. Go to **Runtime** tab → + **New Timeline**
 2. Set name and duration
 3. + **Add Track** for each fixture (or “All Performers”)
 4. + **Add Clip** to assign effects with start time and duration
 5. Clips can overlap — they blend according to their effect's blend mode
-

10. Baking & Playback

Bake

Compiles a timeline into minimal action instructions per performer: 1. Click **Bake** → progress shows frame count and segments 2. Click **Sync** to push instructions to performers via UDP 3. Click **Start** for synchronized NTP-timed playback

Output

- **Action segments:** Sequences of the 19 action types (14 classic + 5 DMX/spatial)
- **LSQ files:** Raw per-pixel RGB data at 40Hz (downloadable as ZIP)
- **Preview data:** 1 color per string per second for emulator

11. Show Preview Emulator

Both desktop and Android include a real-time show preview:

Dashboard Preview

When a show is running, the Dashboard tab shows a live stage preview canvas alongside the performer status table and playback progress bar.

Desktop SPA

The emulator canvas appears on the Runtime tab below the timeline. Shows: - **LED fixtures:** Colored dots along string paths with glow effects - **DMX fixtures:** Beam cone triangles with preview-driven colors - **Aim dots:** Red circles at aim points - **Fixture labels:** Names below each node - **Time counter:** MM:SS elapsed / total

Android App

The `ShowEmulatorCanvas` card shows: - Same LED string dots and DMX beam cones as desktop - Objects rendered as background rectangles - Preview colors update every second during playback

Spatial Field Visualization

During show playback, the runtime emulator renders the active spatial effects moving across the stage: - **Sphere**: translucent colored circle moving along the motion path - **Plane**: translucent horizontal or vertical band sweeping across the stage - **Box**: translucent rectangle at the effect's current position - Effect names shown as labels at their current position - Updates every frame, synced to playback elapsed time

DMX-Only Rigs

The emulator correctly renders DMX-only setups (no LED performers). Static purple beam cones always visible, with live colors when a show is running.

12. DMX Fixture Profiles

Built-in Profiles

PROFILE	CHANNELS	FEATURES
Generic RGB	3	Red, Green, Blue
Generic RGBW	5	Red, Green, Blue, White, Dimmer
Generic Dimmer	1	Intensity only
Moving Head 16-bit	16	Pan, Tilt, Dimmer, Color, Gobo, Prism

Profile Editor — Step-by-Step (#527)

The fixture profile editor maps a DMX channel to what it *does* — this red channel here, that pan channel there — and records any behaviour the fixture firmware expects on each channel (gobo slot ranges, strobe rate curves, colour-wheel slots). Once a profile exists the whole orchestrator can drive that fixture with semantic calls like “set colour to red” or “aim at stage (1150, 2100)” instead of raw DMX.

1. Where to find it

Settings tab → **Profiles** sub-section. The list view shows every built-in and custom profile, filterable by category (par / wash / spot / moving-head / laser / effect). Each row has:

- **Edit** — opens the editor on the selected profile (disabled for built-in profiles; clone first if you want to diverge from one).
- **Clone** — copy a built-in or community profile into your local library under a new id; the copy is editable.
- **Share** — upload a custom profile to the community server (requires an internet connection, rate-limited per IP).
- **Delete** — remove a custom profile (built-in profiles cannot be deleted).

Click **New Profile** to start a fresh editor on a blank profile. You can also reach the editor from any DMX fixture card by clicking the profile name under the fixture's **Edit Profile** button.

2. Top-level fields

- **Name** — operator-visible label shown in fixture cards + the profile picker.
- **Manufacturer** — free text; used for grouping in the Community browser and for dedup matching.
- **Category** — `par`, `wash`, `spot`, `moving-head`, `laser`, `effect`, `other`. Drives the preset-show generator.
- **Channel count** — total DMX slots the fixture uses. Auto-updated as you add channels; can also be set explicitly.
- **Colour mode** — `rgb`, `cmy`, `rgbw`, `rgba`, `single` (monochrome dimmer), `color-wheel-only`, or **hybrid RGB + colour wheel**. Drives how the show engine resolves a requested colour:
 - **RGB family** — the orchestrator writes the requested R / G / B triple to the matching channels.
 - **Colour-wheel-only** — the orchestrator picks the closest wheel slot for the requested RGB via the Euclidean-distance resolver described under Capabilities. Wheel slots without a hex annotation are skipped, so a profile that should match (say) red but doesn't has a missing `color` hex on its red WheelSlot capability.
 - **Hybrid RGB + wheel** — the profile carries BOTH a red/green/blue triple AND a `color-wheel` channel. Since v1.7.83 (#842) the orchestrator drives RGB directly **and** parks the wheel at slot 0 (open / white) automatically, so the wheel's default coloured slot doesn't filter the beam. Pre-v1.7.83 hybrid fixtures often looked "wrong" because the wheel's home slot was tinting an RGB-driven colour; that's now handled at the lowest level (`set_fixture_rgb`) and every render path benefits.

- **Pan range / Tilt range** — maximum mechanical sweep in degrees. Used by mover calibration to normalise DMX→angle.
- **Beam width** — degrees of the beam cone. Used for 3D beam-cone rendering and for marker-coverage prediction.

3. Channels

Every channel has:

- **Offset** — 0-based channel number within the fixture’s address range (not the universe). A 16-channel fixture has offsets 0..15.
- **Name** — operator-facing label. Matches the fixture’s documentation.
- **Type** — the *semantic* role. Common types: `pan`, `pan-fine`, `tilt`, `tilt-fine`, `dimmer`, `red`, `green`, `blue`, `white`, `amber`, `uv`, `color-wheel`, `gobo`, `prism`, `focus`, `zoom`, `frost`, `strobe`, `macro`, `reset`. The type is what downstream code reads when it wants to control “the dimmer” — you can rename the channel but the type is the contract.
- **Bits** — 8 (one DMX slot) or 16 (two slots, coarse at this offset + fine at offset+1). Use 16-bit for pan and tilt if the fixture supports it; everything else is typically 8-bit.
- **Default** — the value the engine writes when no effect is overriding the channel. Leave blank for “set to 0 at idle.” Use a non-zero default for channels the fixture needs lit to function (e.g. a lamp-on macro, shutter-open slot).

4. Capabilities

Each channel can carry a list of capabilities that describe what DMX value ranges mean to the fixture:

- **WheelSlot** — colour or gobo wheel position. Range `[min, max]`, label (`"Red"`, `"Open"`, `"Pattern 3"`), and — for colour wheels — an optional **color** **hex** like `#FF0000`. The orchestrator’s RGB→slot resolver (used by show bake and mover calibration) picks the closest-matching slot by Euclidean distance in RGB space, so every colour-labelled slot needs the hex filled in. Without the hex the RGB pipeline silently falls through to slot 0 (white/open), which is the #624 footgun.
- **WheelRotation** — rotating-wheel range for cycle effects (`"CW cycle fast-slow"`, `"CCW cycle slow-fast"`).
- **WheelShake** — jitter ranges on gobo wheels.
- **ShutterStrobe** — a range with a `shutterEffect` of `"Open"`, `"Closed"`, or `"Strobe"`. The orchestrator’s “open the shutter during calibration” helper walks these caps to find the right DMX value.

- **Prism, PrismRotation, Effect, NoFunction** — same pattern: `range`, `label`, optional type-specific fields.

Each capability row lets you pick the type from a dropdown, set `min`/`max`, add a label, and (for `WheelSlot` on colour wheels) a colour hex swatch.

5. Saving + sharing

- **Save** persists the profile to `desktop/shared/data/dmx_profiles/` (gitignored per-install) and updates the SPA list.
- **Share to Community** uploads the profile JSON to the electricrv.ca server. The server dedups by channel-hash, so submitting a profile someone else already uploaded produces a “this fixture is already covered” response with a link to the existing entry.
- **Export** downloads every custom profile as a single JSON bundle. Use this to transfer a profile library between installs without going through the community server.

6. When to create your own vs import from OFL

- **Import from OFL** first — 700+ fixtures are already there, and importing is one click. The Open Fixture Library volunteers have spent years curating the capability lists.
- **Clone and edit** if the fixture is close to an OFL profile but a channel or two differs (firmware update, mode variant).
- **Create from scratch** only when the fixture is genuinely not in OFL and not in the community. When you’re done, share it so nobody else has to.

Global brightness scaling (#843)

Master brightness reaches DMX fixtures through a per-frame scaling pass at render time, not through the bake. That keeps bakes valid across brightness changes — operators can drag the slider during a show without re-baking — and lets the Android Auto Brightness fast-path drive the master at ~20 Hz with the live envelope of stage audio.

Two cases:

- **Master-dimmer profiles** — when the profile maps a `dimmer` channel, the dimmer byte is scaled. The scaling is gamma-corrected ($\gamma = 2.2$ LUT) so a master setting of 50 % reads on a DMX par the same way it reads on an LED strip (which FastLED gamma-corrects internally).
- **RGB-only profiles** — when there is no `dimmer` channel, the RGB triple is gamma-scaled instead. Wheel-only fixtures fall back to scaling the dimmer (slot indices are

categorical, not intensity).

The orchestrator broadcasts `CMD_SET_BRIGHTNESS` to every LED child on every change, and re-sends the current value to a child the first time it appears in PONG after a power cycle, so a freshly booted child doesn't display its first show frame at full intensity.

Aim cone clamping (#803)

Aim requests on a moving head are clamped to the fixture's reachable cone, not rejected. If a Track action or `/api/mover/<fid>/aim` target lands outside the head's pan / tilt sweep, the orchestrator slides the aim along the cone rim toward the target instead of returning a 400 error. Operators see the head reach as far as it mechanically can; the 3D viz cone reflects the clamped aim, not the rejected one. Pre-v1.7.30 the operator-direct route would refuse the write, leaving the rig in a half-aimed state.

Legacy quick-reference

Settings tab → **Profiles** → **New Profile** or **Edit**: - Define channels with name, type (red/green/blue/dimmer/pan/tilt/etc.), default value - Set beam width, pan/tilt range for moving heads - Import from Open Fixture Library (OFL) JSON format

Browsing the Open Fixture Library

Click **Search OFL** in Settings → Profiles to access 700+ fixtures from the [Open Fixture Library](#):

Search: Type a fixture name, manufacturer, or keyword → results show with Import buttons.

Browse by Manufacturer: Click **Manufacturers** to see all brands with fixture counts. Click a manufacturer to see all their fixtures. Click **Import All** to import every fixture from that manufacturer at once.

Bulk Import: From search results, click **Import All** to import all matching fixtures. From a manufacturer page, click **Import All** for the entire brand catalog.

Multi-mode fixtures create one SlyLED profile per mode automatically.

Community Fixture Library

Share and discover profiles with other SlyLED users:

1. **Browse:** Click **Community** in Settings > Profiles to search, view recent, or popular
2. **Download:** Click Download — imported to your local library immediately
3. **Share:** Click **Share** on any custom profile to upload to the community
4. **Dedup:** Server detects duplicates by channel fingerprint (same channels = same fixture)
5. **Unified search:** When adding a DMX fixture, search queries Local + Community + OFL at once

Community server: <https://electricrv.ca/api/profiles/>

Import/Export

- **Community:** Share/download profiles with other users
 - **Search OFL:** Browse, search, and bulk import from the Open Fixture Library
 - **Paste OFL:** Paste raw OFL JSON for offline fixtures
 - **Import Bundle:** Load previously exported profile pack
 - **Export:** Download all custom profiles as JSON
 - **Built-in profiles** cannot be edited or deleted
-

13. Preset Shows

Preset shows are a one-click way to put any rig into a polished look — useful for soundcheck, pre-show ambience, or a quick reset between cues. Open them from **Runtime → Load Show → Presets**.

The show generator inspects the rig at install time (LED strips, moving heads, camera nodes) and adapts each theme to what's actually on the stage. Themes that include moving-head choreography auto-classify any DMX fixture with `panRange > 0` and `tiltRange > 0` as a candidate, then emit either coordinated sweeps (stage-coordinate motion via `ptStartPos` / `ptEndPos`) or live target-tracking via a Track action.

Available themes

PRESET	DESCRIPTION	MOVERS DO
Rainbow Up	Rainbow plane rising floor to ceiling	Stage-coord sweep up the Z axis (#837)
Rainbow Across	Rainbow sphere sweeping stage-left to stage-right	Stage-coord sweep along X
Slow Fire	Warm fire effect on every fixture	Static beam, dimmer flicker
Disco	Pastel twinkle sparkles	Mid-stage stage-coord chase
Ocean Wave	Blue wave sweep with teal wash	Slow front-to-back sweep
Sunset Glow	Warm breathe with golden sweep	Static aim, breathing dimmer
Police Lights	Red strobe with blue flash sweep	Strobe + slow side-to-side
Starfield	White sparkles on dark background	Static beam, dimmer twinkle
Aurora Borealis	Green curtain with purple shimmer	Slow front-to-back sweep
Aurora Curtain <i>(new in v1.7.83)</i>	Coordinated travelling curtain — every mover rides the same ribbon target at a phase offset, so the curtain visibly travels the rig instead of every head moving in unison. Includes a sparkle layer and fade-in / fade-out brackets so the look opens and closes cleanly.	Track a ribbon target, ping-pong on the chosen axis
Spotlight Follow Person	Warm orb that camera-tracked people inherit	Track action — heads chase detected person
Concert Wash	Magenta flood + amber sweep	Stage-coord sweep, no tracking
Figure Eight	Crossing orbs — heads trace X paths	Track action — heads chase crossing patrol props
Thunderstorm	Lightning strikes from the rig top-down	Strobe-style stage-coord bolts (Z-down, #837)
Dance Floor	Fast orbiting spots	Stage-coord chase, no tracking

Track-action vs sweep

The description column tells you whether the heads chase a target (“Track action”) or follow a pre-baked sweep path (“stage-coord sweep”). Pre-v1.7.83, several themes promised tracking in their description but emitted only a sweep — fixed in #837 so the table is honest now: only **Spotlight Follow Person**, **Figure Eight**, and **Aurora Curtain** create a Track action when installed.

Customising an installed preset

Installing a preset writes it into the regular Actions / Timeline / Objects state, so every part of it is editable in place:

- **Recolour a sweep** — open the action under **Actions**, change RGB. The bake re-renders next time the timeline plays; no need to re-install the preset.
- **Scope a Track action to specific fixtures** — open the Track action’s Advanced expander (chapter 8), set `trackFixtureIds` to the heads you want, leave it empty to scope to every mover.
- **Adjust the tracker cycle** — Cycle Time (ms) in the Advanced expander controls how often a head jumps to a new target when there are more targets than heads.
- **Replace the patrol target** — for Aurora Curtain or Figure Eight, the underlying patrol object is on the **Layout / Objects** tab. Change its axis, speedPreset, or pattern; the next playback frame picks up the new motion.

Behaviour outside playback

Preset shows respect the same idle-behaviour contract as any other timeline (chapter 17 → “Idle behaviour”): the rig parks at home and the lamp closes when the show ends, claim is released, or the operator stops playback. The Aurora Curtain ribbon’s patrol object auto-stops at timeline end since it’s tagged `patrolMode: on-demand` when the preset installs it (chapter 6).

14. Camera Nodes

Camera nodes are Orange Pi or Raspberry Pi single-board computers with **USB cameras**. They provide live snapshots and AI-powered object detection for stage setup, mover calibration, and live person tracking via the Spotlight Follow Person preset.

The current production camera firmware is **v1.6.3** (paired with orchestrator v1.7.83). Older deployed nodes can be updated in place from the Firmware tab — the SSH-deploy path picks up the new firmware bundle from `dist/camera-firmware-v1.6.3.zip` and replaces the camera service without rebooting the SBC.

Note: Only USB cameras are supported. Pi CSI ribbon cameras (e.g. Pi Camera Module, Freenove FNK0056) are not supported. Use USB webcams instead. Compatibility matrix for SBCs and USB sensors lives in `docs/SUPPORTED_HARDWARE.md` — Orange Pi 4A is the primary target; RPi 3B+ / 4 / 5 and Orange Pi Zero 3/5 are confirmed working.

Adding a Camera Node

1. Flash an Orange Pi with the supported OS image
2. Connect it to the same WiFi network as the orchestrator
3. In the **Firmware** tab, configure SSH credentials (default: `root` / `orangeypi`)
4. Click **Scan for Boards** to find the device on the network
5. Click **Install** to deploy the camera firmware via SSH+SCP

Camera Config Page

Each camera node serves a local web interface at `http://<camera-ip>:5000/config`: - **Dashboard** — board info, per-camera cards with live capture and detection - **Settings** — device name, reboot, factory reset

Snapshots

Click **Capture Frame** on any camera card to take a JPEG snapshot. Uses OpenCV for fast capture with fswebcam fallback.

Object Detection

Click **Detect Objects** (purple button) to run YOLOv8n AI detection on the current camera frame: - Bounding boxes with labels and confidence percentages are drawn on a canvas overlay - **Threshold slider** (0.1-0.9) — filter by detection confidence - **Resolution** (320/640) — lower is faster, higher is more accurate - **Auto checkbox** — continuously detect every 3 seconds - Typical latency: ~500ms capture + ~500ms inference on Orange Pi 4A

Detection requires the YOLOv8n ONNX model (`models/yolov8n.onnx`, 12 MB), which is uploaded automatically during firmware deployment.

Camera Deploy

The deploy process (from the **Firmware** tab) uploads all camera files via SCP: -

`camera_server.py`, `detector.py`, `requirements.txt`, `syled-cam.service` - `models/yolov8n.onnx` (detection model) - Installs system packages (`python3-opencv`, `python3-numpy`, `v4l-utils`) - Installs Python packages (`flask`, `zeroconf`, `onnxruntime`) - Sets up the `syled-cam` systemd service for auto-start on boot - Shows version comparison and supports force-reinstall

Multi-Camera Support

Each node can host multiple USB cameras. The firmware auto-detects connected cameras and filters out internal SoC video nodes. Each camera gets its own card in the config page with independent capture and detection controls.

Environment Scanning

The **Scan Environment** button on the Layout toolbar captures a 3D point cloud of the physical space: 1. Each positioned camera captures a frame and runs depth estimation 2. Pixels are back-projected to 3D using camera FOV and depth 3. Point clouds from all cameras are merged into stage coordinates 4. **Surface analysis** identifies floor, walls, and obstacles (pillars, furniture) 5. Detected surfaces can be automatically created as named stage objects

The point cloud can be viewed as colored dots in the 3D viewport (toggle with the point cloud button). This gives a visual map of the physical environment that the lights will illuminate.

Per-Camera Fixtures

Each USB camera sensor on a camera node registers as a **separate fixture** in the layout. A node with 2 cameras creates 2 fixtures, each with: - Its own position on the stage (independently placeable) - Its own FOV and resolution - Its own rest direction vector (cyan arrow)

Tracking Configuration

Each camera fixture has per-camera tracking settings accessible from the **Edit** dialog on the Setup tab. These control what the camera detects and how it behaves during live tracking.



Camera edit with tracking config

Camera edit with tracking config

Detect Classes — Multi-select the object types to track. The YOLOv8n model supports 80 COCO classes; 16 stage-relevant classes are available:

CATEGORY	CLASSES
People	Person
Animals	Cat, Dog, Horse
Props	Chair, Backpack, Suitcase, Sports Ball, Bottle, Cup, Umbrella, Teddy Bear
Vehicles	Bicycle, Skateboard, Car, Truck

By default only **Person** is selected. Adding more classes has zero performance impact — YOLO always evaluates all classes in one pass and filters afterward.

Parameters:

PARAMETER	DEFAULT	RANGE	DESCRIPTION
FPS	2	0.5-10	Detection frames per second. Higher = more responsive but more CPU on the camera node.
Threshold	0.4	0.1-0.95	Minimum confidence to accept a detection. Lower = more sensitive but more false positives.
TTL (s)	5	1-60	Seconds before a lost track expires and its stage marker is removed.
Re-ID (mm)	500	50-5000	Maximum distance to match a new detection to an existing tracked object.

Starting tracking: Click the **Track** button on the Setup tab (next to Snap) or in the fixture edit modal on the Layout tab. The camera node begins continuous detection using your configured classes and parameters. Detected objects appear as labeled markers in the 3D viewport.

Mover Calibration

The mover calibration wizard builds an interpolation grid that maps every stage position to the exact pan/tilt angles required for a DMX moving head. A positioned camera node is required.

Prerequisites: - At least one camera node positioned on the Layout tab - Art-Net engine running (`POST /api/dmx/start`) - Moving head fixture placed on the layout with its profile configured

Starting calibration: 1. Go to the **Layout** tab and double-click a DMX moving head fixture 2. Click the **Calibrate** button in the fixture edit dialog 3. Choose a beam color — options are Green, Magenta, Red, Blue (pick one that contrasts with your stage) 4. Click **Start Calibration** — the wizard takes over automatically

What happens automatically: 1. **Discover** — the head sweeps through a coarse pan/tilt grid; the camera detects where the beam lands on the stage floor 2. **Map visible region** — the pan/tilt range that keeps the beam within the camera's field of view is identified 3. **Build interpolation grid** — the head systematically samples points across the visible region; at each point the camera records the exact stage coordinates

Progress: A real-time progress panel shows the current phase, percentage complete, and a live thumbnail from the camera.

Result: The interpolation grid is saved to the fixture and used automatically by the Track action and any Pan/Tilt Move actions to convert stage-space coordinates into hardware pan/tilt values.

Tip: Run calibration in dim ambient lighting so the beam is clearly visible to the camera. Use the **Beam Color** option that gives the highest contrast against your floor surface.

Fixture Orientation Test

Before running full calibration, use the orientation test to confirm pan and tilt are wired in the expected directions. Incorrect orientation causes calibration to converge on wrong positions.

Running the test: 1. Double-click a DMX moving head on the Layout tab to open the fixture edit dialog 2. Click **Orientation Test** (below the channel map) 3. The fixture moves through four probe positions: pan left, pan right, tilt up, tilt down 4. Watch the physical beam and compare with the on-screen arrows showing expected direction

Interpreting results: | Observation | Action | |-----|-----| | Beam matches arrows | Orientation is correct — proceed to calibration | | Pan moves opposite direction | Enable **Invert Pan** in the fixture settings | | Tilt moves opposite direction | Enable **Invert Tilt** in the fixture settings | | Pan and tilt axes are swapped | Enable **Swap Pan/Tilt** in the fixture settings |

Saving: After adjusting orientation flags, click **Save** in the fixture edit dialog. The flags are stored with the fixture and applied automatically during all subsequent calibration and playback.

15. Firmware & OTA Updates

The Firmware tab is the operator's single window onto every flashable device on the rig: LED performers, the DMX-bridge, the gyro puck, and the camera nodes. Every flashable device reports its current firmware version up to the orchestrator on each PING/PONG cycle, so a stale device shows as "outdated" within seconds of the orchestrator booting.

Current production versions (orchestrator v1.7.83)

DEVICE	TRACK	CURRENT	CHANNEL
Orchestrator (Windows / macOS)	app	v1.7.83	installer (<code>SlyLED-Setup.exe</code>)
Android operator app	app	matches orchestrator track	sideload APK from <code>dist/slyled-android.apk</code>
LED Performer (ESP32)	<code>child-led-esp32</code>	v7.5.11	OTA
LED Performer (D1 Mini)	<code>child-led-d1mini</code>	v7.5.10	OTA
LED Performer (Giga child)	<code>child-led-giga</code>	v7.5.2	OTA
DMX Bridge (ESP32)	<code>dmx-bridge-esp32</code>	v7.5.20	OTA
DMX Bridge (Giga R1)	<code>dmx-bridge-giga</code>	v7.5.20	OTA
Parent firmware (Giga R1)	<code>parent-giga</code>	v7.5.24 <i>(on hold — desktop orchestrator is the recommended runtime)</i>	USB only
Gyro Controller (ESP32-S3)	<code>gyro-esp32s3</code>	v1.2.8	OTA
Camera node (Linux SBC)	<code>camera-node</code>	v1.6.3	SSH deploy from Firmware tab

The Firmware tab queries `firmware/registry.json` to know what the “current” version is, so this table is regenerated automatically every release; the operator never has to keep it in their head.

USB Flash

1. Go to the **Firmware** tab.
2. Click the **USB Flash** card. The dropdown lists every binary the registry knows about for boards that flash over USB (LED ESP32 / D1 Mini / Giga child / DMX bridge variants / Gyro Controller).
3. Plug the target board in and pick its COM port from the second dropdown.
4. Click **Flash** — progress shows percentage and a final “verification OK” before the board reboots into the new firmware.

The Gyro Controller (ESP32-S3) ships with USB-CDC serial in the firmware. If a wedged build leaves the puck unable to enumerate over USB, hold the **BOOT** button while plugging in to enter the manual ROM bootloader; the Firmware tab then re-flashes through esptool's recovery path.

OTA (Over-the-Air)

1. Set WiFi credentials on the Firmware tab — these get pushed to every newly-flashed device.
2. Click **Check for Updates**. The tab shows a per-device comparison: reported version → registry version, with an **Update** button on anything outdated.
3. Click **Update** on any outdated performer. Mid-flash status comes back live; the device reboots automatically after verification.
4. New since v1.7.83: when a registry SHA-256 mismatches the on-disk binary (a download mid-update or a hand-edited registry), the orchestrator falls back to the GitHub release for that board's `releaseTag` rather than refusing the flash.

The diagnostic / development gyro builds (`esp32s3-gyro-test-firmware.bin`) are deliberately hidden from the operator OTA UI — the Firmware tab only offers production builds. The diagnostic binary is still present in `dist/` for engineers running paired-board debugging.

Firmware Registry

`firmware/registry.json` is the single source of truth for what version the orchestrator believes ships with each release. Each entry carries:

- `id` and `name` for the OTA UI label.
- `version` (3-part semver) — what the operator's device should be running.
- `releaseTag` and `releaseAsset` — the GitHub release tag and the asset filename inside it, used by the OTA fallback.
- `sha256` — verification hash that the orchestrator checks before and after flashing.

Editing `registry.json` by hand is not recommended; `build_release.ps1` keeps it in sync with the actual binary hashes on every release. The Firmware tab refreshes the registry from GitHub on demand from the **Refresh** button so a freshly-pulled installer immediately sees the versions corresponding to its release tag.

16. System Limits

RESOURCE	TESTED	RECOMMENDED MAX
DMX fixtures	120	500+
LED performers	12	50
Total fixtures	132	500+
Universes	4	32,768 (Art-Net)
LEDs per string	65535	uint16 addressing
Strings per child	8	Protocol constant
Timeline clips	50	200+
Preset shows	14	Built-in (expandable)
API response (132 fixtures)	< 1ms	Sub-millisecond
Memory (132 fixtures)	46 MB	Flat scaling
Network (132 fixtures)	221 KB	Per test cycle

See `docs/STRESS_TEST.md` for full benchmark data.

17. Troubleshooting

The table below covers the symptoms operators most often raise. Every row points to the orchestrator version where the underlying behaviour was last touched, so an operator running an older build can decide whether to update or work around.

PROBLEM	WHAT YOU'LL SEE	FIX OR VERSION
Runtime view empty	The 3D Runtime tab shows the stage but no fixtures.	Check that fixtures are positioned in the Layout tab. DMX-only rigs render correctly since v1.7.30.
Beam cone wrong direction	The 3D viz cone aims at the wrong wall.	Beam direction comes from the fixture's <code>rotation = [rx, ry, rz]</code> in stage space. Z is up; rx > 0 aims down. See chapter 4 for the full convention.
3D viz cone disagrees with the physical head	Cone in the viz points stage-left but the moving head is aimed stage-right.	Fixed in v1.7.52 (#806/#809): the canonical aim vector is the source of truth and the physical IK derives from it. If you still see the disagreement on v1.7.52+, re-save the fixture's Home and Secondary in the Set Home wizard.
Calibrate-end pan jump	Pressing release on calibrate snaps the head to a different pose than the puck was reporting.	Fixed in v1.7.52 (#805). Pre-fix the legacy IK fallback was capturing the wrong aim vector at release time. Operators on v1.7.52+ who still see a jump should report it with the gyro-puck firmware version (must be $\geq$ v1.2.4).
Press Start blinks back to "start" on the puck	Operator presses Start after a WiFi gap, the puck UI flashes claim-acknowledge for a frame, then reverts to IDLE while the orchestrator holds an orphan claim.	Fixed in v1.7.83 (#812 / #813 / #825). Press-Start now uses a 16-bit nonce + CLAIM_ACK, with HB_REP heartbeats to reconcile divergent state. If you see the symptom on v1.7.83+, check that the puck firmware is $\geq$ v1.2.7 (registry will warn).
Auto Brightness has no effect on the lights	The Android Auto Brightness UI shows the master sliding with the music, but DMX heads and LED strips don't dim.	Fixed in v1.7.83 (#843). The fast-path POST now broadcasts <code>CMD_SET_BRIGHTNESS</code> to LED children and gamma-scales DMX dimmer / RGB at render time. Operators on older builds can fall back to the manual Settings → Global Brightness slider until they update.

PROBLEM	WHAT YOU'LL SEE	FIX OR VERSION
Looping playlist blacks out between iterations	A single-item or multi-item playlist set to Loop All flashes everything to zero for one frame at every wrap.	Fixed in v1.7.83 (#840). Single-item loops route through the modulo-wrap playback path; multi-item loops pass <code>is_final=False</code> to suppress the natural-end blackout sweep until the playlist actually stops.
Track action blacks out movers in unrelated timelines	A timeline that doesn't reference a particular Track action still has its movers go dark whenever that action exists in the action library.	Fixed in v1.7.83 (#835). Track actions now only evaluate on timelines that reference them; orphan actions stay dormant.
Show preset says "moving heads track / follow / chase X" but they don't	A theme description promises tracking, but the rig just sweeps.	Fixed in v1.7.83 (#837). Theme descriptions now match the actual implementation: only Figure Eight and Spotlight Follow Person emit a Track action; the others sweep.
Colour Wheel field appears in DMX Scene action editor	The action editor's DMX Scene / PT-Move / Gobo Select panes had a "Colour Wheel" input that didn't do what the operator expected on hybrid RGB+wheel fixtures.	Fixed in v1.7.83 (#841 / #842). The Colour Wheel slot is now type-17 only; the bake/render layer derives the slot from RGB via <code>rgb_to_wheel_slot</code> for every other action type. A one-shot migration strips stale <code>colorWheel: 0</code> fields on first start of v1.7.83+.
3D viewport not rendering	Black canvas where the stage should be.	Use Chrome / Firefox / Edge with WebGL support. Check <code>chrome://gpu</code> for hardware acceleration.
Performers not syncing	A child shows offline in Setup but is powered up.	Check that the orchestrator and the child are on the same WiFi subnet. The Setup tab's Refresh rescans via mDNS + UDP broadcast.
Canvas wrong size	The Layout canvas is much smaller or larger than the room.	Stage dimensions (Settings → Stage) drive canvas size: <code>canvasW = stage.w × 1000</code> . Adjust stage width/height in metres rather than canvas pixels.

PROBLEM	WHAT YOU'LL SEE	FIX OR VERSION
OTA flash refused with SHA mismatch	Firmware tab refuses to update with <code>sha256 mismatch</code> .	Fixed in v1.7.61 (#814). The orchestrator now falls back to the GitHub release for the registered <code>releaseTag</code> when the on-disk binary disagrees with the registry. If you still see this, click Refresh on the Firmware tab to re-fetch <code>registry.json</code> from GitHub.
Gyro stale-reason latch never clears	"Connection lost" stays on a puck status row even after the puck resumes streaming.	Fixed in v1.7.62 (#821) and again in v1.7.63 (#823). Press-Start clears the remote <code>stale_reason</code> ; cache self-destructs on a transient read failure.

If you hit something not in this table, the orchestrator's log (Settings → Logging → enable file logging) captures every UDP send and DMX render decision tagged by issue number — open an issue on GitHub with the relevant section attached and a description of what the rig was doing at the moment.

18. Examples

Example A: Camera Tracking — Moving Heads Follow a Person (#376)

Make DMX moving heads automatically follow people detected by a camera.

Prerequisites: - At least one camera node online (Firmware tab → deploy + verify) - At least one DMX moving head fixture placed on the Layout tab - Moving head profile configured with pan/tilt range - Art-Net/sACN engine running (Settings → DMX → Start) - Mover calibration completed (see Example C) for accurate aiming

Steps:

1. **Verify hardware** — Open the Setup tab. Confirm your movers show green status and camera nodes show online. If cameras are offline, check WiFi and deploy firmware from the Firmware tab.

2. **Start camera tracking** — Click the **Track** button on the Setup tab (next to Snap), or go to the Layout tab, click a camera fixture, and click the **Track** button in the edit modal. The camera node begins running YOLO detection using the classes and parameters configured in the camera's tracking settings (see [Tracking Configuration](#)). Detected objects appear as labeled markers in the 3D viewport.
3. **Create a Track action** — Go to the Actions tab. Click + **New Action**.
 - **Name:**
 - **Type:** (last option in the dropdown)
 - **Colour:** Pick the beam color (e.g. red for a spotlight)
 - Leave **Target Objects** empty — this means “follow ALL detected people”
 - **Cycle Time:** 2000 ms (how fast heads switch if cycling)
 - Check **Fixed assignment** if you want strict 1:1 (head 1 = person 1, extras ignored)
 - Click **Save Action**
4. **Create a timeline** — Go to the Shows tab. Click + **New Timeline**, name it “Person Tracking”, set duration to 600s, enable **Loop**. The timeline can be empty — Track actions evaluate globally during any playback.
5. **Start playback** — Click **Bake**, then **Start**. The 40 Hz DMX playback loop begins. The Track action reads all moving temporal objects (detected people), computes pan/tilt for each head, sets dimmer and color, and sends Art-Net packets to the bridge.
6. **Test** — Walk in front of the camera. Within 2 seconds a pink person marker appears in the 3D viewport. The moving heads should light up in your chosen color and aim at you.

Assignment behavior:

PEOPLE IN VIEW	WITH 2 MOVING HEADS
1 person	Both heads aim at the same person
2 people	One head per person (1:1)
3+ people (cycling)	Heads cycle through people every 2s
3+ people (fixed)	First 2 tracked, 3rd ignored

Troubleshooting:

PROBLEM	SOLUTION
No person markers in 3D	Check camera node status — is tracking running? Try a manual Scan to verify detection works.
Person detected but heads don't move	Check Art-Net engine is running. Check mover calibration. Verify timeline playback is active.
Heads light up but aim at wrong position	Run mover calibration (Example C). Without calibration, the system uses geometric estimates which may be inaccurate.
Heads respond with delay	Normal — detection runs at 2 fps with ~1s capture latency. Temporal objects have 5s TTL.

Example B: Mover Tracking with Spatial Effects (#379)

Make moving heads follow a virtual object sweeping across the stage — no camera required. This example walks through the complete workflow from stage setup to live 3D preview with animated beam cones.

Prerequisites: - SlyLED orchestrator running (Windows or Mac) - No physical hardware required — this example runs entirely in the emulator

Part 1 — Stage and Fixture Setup

1. **Set stage dimensions** — Open the Settings tab. Under **Stage**, enter the dimensions of your performance area:
 - Width: 6000 mm (6 m)
 - Height: 3000 mm (3 m)
 - Depth: 4000 mm (4 m)
 - Click **Save**. The 3D viewport will resize to match these dimensions.
2. **Create a DMX profile** — Go to Settings → **Profiles** → click **New Profile**. This defines the channel layout of your moving head:
 - **Name:**
 - **Beam Width:** 8 (degrees — narrow beam for visible tracking)
 - **Pan Range:** 540, **Tilt Range:** 270
 - **Channels:** Add 6 channels in this order:
 - Ch 0: Pan (16-bit) — pan coarse

- Ch 1: Pan Fine — pan fine (auto-linked)
- Ch 2: Tilt (16-bit) — tilt coarse
- Ch 3: Tilt Fine — tilt fine (auto-linked)
- Ch 4: Dimmer
- Ch 5: Red, Ch 6: Green, Ch 7: Blue
- Click **Save Profile**



Profile editor with narrow spot configuration

Profile editor with narrow spot configuration

3. **Add two movers** — Go to the Setup tab. Click + **Add Fixture** twice to create two DMX moving heads:

- **Fixture 1:** Name: (Stage Left), Universe: 1, Start Address: 1, Profile:
- **Fixture 2:** Name: (Stage Right), Universe: 1, Start Address: 14, Profile:

Both fixtures appear in the Setup table with purple “DMX” badges and the profile name.

Part 2 — 3D Layout and Spatial Effect

4. **Position movers on the truss** — Switch to the Layout tab. In the sidebar, you’ll see both movers listed as “unplaced.” Drag each one into the 3D viewport:

- **Mover SL:** Position at X: 1500, Y: 0, Z: 2800 (stage left, on truss). Set rotation to tilt: -30, pan: -15.
- **Mover SR:** Position at X: 4500, Y: 0, Z: 2800 (stage right, on truss). Set rotation to tilt: -30, pan: 15.

Switch to 3D view to confirm both movers are elevated on the truss and aimed downward toward the stage floor. The beam cones should be visible as translucent triangles.



Layout tab — 3D view with two movers positioned on truss

Layout tab — 3D view with two movers positioned on truss

5. **Create a spatial effect** — Go to the Actions tab. Click + **New Action:**

- **Name:**

- **Type:** Spatial Effect
- **Shape:** Sphere
- **Radius:** 800 mm
- **Color:** Green (0, 255, 0)
- **Motion Start:** X: 1000, Y: 2000, Z: 0 (stage left, mid-depth, floor level)
- **Motion End:** X: 5000, Y: 2000, Z: 0 (stage right, same depth and height)
- **Duration:** 8 seconds
- **Easing:** Linear
- Click **Save Action**

This creates a green sphere of light that sweeps from stage left to stage right over 8 seconds. When applied to moving heads, they will track the sphere's center position.



Actions tab with Sweep Green spatial effect configured

Actions tab with Sweep Green spatial effect configured

Part 3 — Timeline, Bake, and Playback

6. **Create a timeline** — Go to the Shows tab. Click + **New Timeline:**

- **Name:** Mover Tracking Demo
- **Duration:** 20 seconds
- **Loop:** Enabled
- Add a track targeting **All Performers**
- Add a clip referencing the **Sweep Green** effect, starting at 0s with 8s duration



Shows tab with timeline containing the spatial effect clip

Shows tab with timeline containing the spatial effect clip

7. **Bake the timeline** — Click the **Bake** button. The bake engine computes per-fixture pan/tilt angles for each time slice:

- For each 25ms frame, it calculates the sphere's position along the motion path
- For each mover, it computes the pan/tilt angles needed to aim at that position
- Dimmer is set to 255 and color channels are set to green
- Wait for "Bake complete" confirmation (typically < 1 second)

8. **Start playback and verify** — Switch to the Runtime tab. Click **Start:**

- The 3D viewport shows both beam cones animated in real-time
- At T=0s, both beams aim at the starting position (stage left)
- As the effect sweeps, the beams track the green sphere across the stage
- At T=8s, both beams have followed the sphere to stage right
- The timeline loops, and the sweep restarts

 Runtime — beam cones at start position (T=0s)

 Runtime — beams tracking mid-sweep (T=5s)

 Runtime — beams at end position (T=10s)

What to look for: - Both beam cones should be green (matching the effect color) - The cones should move smoothly from left to right - The beam intensity (opacity) should be > 0 during the sweep, indicating active output - If beam cones don't appear, ensure fixtures are positioned in the Layout tab and the timeline is baked

Variations: - Change the spatial effect shape to **Plane** for a wall of light sweeping across - Add a second effect on a separate track with different timing for crossing patterns - Try the **Figure Eight** preset show (Runtime → Load Show) for a ready-made crossing pattern

Example C: Manual Mover Calibration (#381)

Calibrate a moving head so the system knows exactly where its beam lands for any pan/tilt position. This two-part process first discovers the beam's visible range (pan/tilt grid) and then builds a light map that maps every pan/tilt position to real stage coordinates.

Prerequisites: - At least one camera node online and positioned on the Layout tab - Camera calibration complete — the camera must have a valid stage map (see Example D) - Moving head fixture added in Setup and positioned on the Layout tab - Art-Net/sACN engine running (Settings → DMX → Start) - Dim ambient lighting — the beam must be clearly visible to the camera against the floor - The beam should be aimed at the floor within the camera's field of view, not directly at the camera

Part 1 — Pan/Tilt Discovery and Grid Calibration

1. **Open the calibration panel** — Go to the Layout tab. Double-click the moving head fixture you want to calibrate. In the edit dialog, click the **Calibrate** button. The calibration wizard opens showing the fixture name, current calibration status (if any), and available calibration modes.



Calibration panel before starting — shows fixture name and calibration options

Calibration panel before starting — shows fixture name and calibration options

2. **Choose beam color** — Select a color that contrasts well with your floor surface:

- **Green** works best on dark floors (wood, dark carpet)
- **Magenta** works best on light floors (white, concrete)
- **Red** or **Blue** are alternatives if the default choices blend with your environment
- The color matters because the camera uses color filtering to isolate the beam from ambient light

3. **Run discovery** — Click **Start Calibration**. The system runs an automatic discovery sequence:

- **Phase 1 — Coarse grid scan:** The fixture sweeps through ~40 pan/tilt positions (8 columns x 5 rows) across its full range. The camera watches for the beam appearing on the floor after each move.
- **Phase 2 — Fine refinement:** Once the beam is found, the system spirals outward from that position to refine the exact center of the visible region.
- Discovery typically completes in 30-60 seconds. The progress indicator shows “Discovering...” with the current scan position.



Discovery in progress — coarse grid scan with camera watching for beam

Discovery in progress — coarse grid scan with camera watching for beam

4. **BFS mapping** — After discovery, the system automatically maps the full visible region:

- Starting from the discovered beam position, it steps in 4 directions (up/down/left/right in pan/tilt space)
- At each position, the camera captures a frame and detects the beam centroid
- The system records the beam’s pixel position and converts it to stage millimeters using the camera’s homography
- Mapping stops at boundaries where the beam leaves the camera’s field of view or falls off the stage
- Collects up to 60 sample positions, typically completing in 2-3 minutes
- The system uses adaptive settle times (0.8-2.5s) per move and double-capture verification to ensure the beam has stopped before recording

5. **Grid build and review** — The collected samples are compiled into a bilinear interpolation grid:

- The calibration summary displays:
 - **Sample count:** Number of successfully detected positions (aim for 30+)
 - **Pan range:** Normalized range (e.g., 0.15-0.85 means the beam is visible across 70% of pan range)
 - **Tilt range:** Normalized range
 - **Grid density:** How finely the grid was sampled
- The grid enables fast forward lookup: given a (pan, tilt) value, compute the stage (X, Y) where the beam lands



Grid calibration complete — summary showing sample count, pan/tilt range, and grid density

Grid calibration complete — summary showing sample count, pan/tilt range, and grid density

Part 2 — Light Map Calibration (stage coordinates to pan/tilt lookup)

6. **Build the light map** — Click **Build Light Map**. This extends the calibration by sweeping a systematic 20x15 grid across the discovered visible region:

- For each grid position, the fixture moves to the pan/tilt value
- The camera detects the beam and records the exact stage X/Y/Z where it lands
- This builds a comprehensive (pan, tilt) → (stageX, stageY, stageZ) lookup table
- Progress shows as “Building light map... N/300” with real-time updates
- Typical completion time: 5-10 minutes for a full 20x15 grid



Light map build in progress — systematic sweep with stage coordinate mapping

Light map build in progress — systematic sweep with stage coordinate mapping

7. **Verify inverse lookup** — After the light map is built, use the **Aim** button to test the inverse mapping:

- Enter a target stage position (e.g., center stage: X=3000, Y=2000, Z=0)
- Click **Aim** — the system uses inverse-distance weighted interpolation of the 4 nearest light map samples to compute the exact pan/tilt values
- The fixture moves to the computed position

- Verify visually that the beam lands on (or very near) the target point on stage
- Try 3-4 different targets across the stage to confirm accuracy
- Good calibration should place the beam within 100-200mm of the target at typical stage distances



Aim verification — beam aimed at target stage position using calibrated light map

Aim verification — beam aimed at target stage position using calibrated light map

8. **Save calibration** — Calibration data is automatically saved with the fixture. The light map and grid data persist across sessions and are included in project file exports (.slyshow).

- Track actions use the light map to aim at detected people
- Pan/Tilt Move actions use it for smooth interpolated sweeps
- The 3D viewport uses it to render accurate beam cone directions

Manual calibration (alternative — no camera required):

If automated calibration isn't available (no camera, or camera can't see the beam), use the manual calibration wizard:

1. Layout tab → double-click mover → click **Manual Calibrate**
2. **Define marker positions** — Add 4-6 physical markers at known stage positions. Enter each marker's X, Y, Z coordinates (in mm). Spread markers across the stage: front-left, front-right, back-center at minimum.
3. **Jog to each marker** — For each marker, use the pan/tilt sliders to manually aim the beam until it lands exactly on the physical marker. Click **Record** to save the (pan, tilt) → (stageX, stageY, stageZ) sample.
4. **Add at least 4 samples** spread across the stage for a good affine fit. More samples (6+) improve accuracy, especially at stage edges.
5. Click **Compute** — the system fits a 3D affine transform from your samples:
 - $\text{pan} = a1 \cdot \text{stageX} + b1 \cdot \text{stageY} + c1 \cdot \text{stageZ} + d1$
 - $\text{tilt} = a2 \cdot \text{stageX} + b2 \cdot \text{stageY} + c2 \cdot \text{stageZ} + d2$
 - The affine transform extrapolates beyond calibrated points for full-stage coverage

When to re-calibrate: - Fixture physically moved to a new position or angle - Venue change (different stage dimensions or floor surface) - After firmware update that changes pan/tilt range or motor behavior - If aim accuracy degrades over time (motor drift) - After changing the fixture's mounting orientation (upright vs. inverted)

Example D: Camera Calibration with ArUco Markers (#380)

Calibrate a camera so pixel coordinates can be mapped to real stage positions. This is a prerequisite for beam detection, person tracking, and mover calibration — without it, the system cannot convert what the camera sees into real-world stage millimeters.

Prerequisites: - Camera node online and reachable on the network (deploy firmware from the Firmware tab if needed) - Camera fixture registered in the system (Setup tab → Discover, or Settings → Cameras → add manually) - Camera fixture placed on the Layout tab at its physical position - A printer to print the ArUco marker sheet (standard A4/Letter paper) - A tape measure to record marker positions on stage - The camera must have a clear view of the stage floor where markers will be placed

Part 1 — Prepare and Place ArUco Markers

1. **Print ArUco markers** — Go to Settings → Cameras. Click the **Print ArUco Markers** button. A modal opens with 6 printable ArUco 4x4 markers (IDs 0-5), each 150mm x 150mm:
 - Click **Download** or use the browser's print dialog to print the marker sheet
 - Print at 100% scale (no scaling/fit-to-page) — the physical size must match the expected 150mm for accurate calibration
 - Markers can be printed on regular white paper, but card stock is more durable



ArUco marker print dialog — 6 markers ready to print

ArUco marker print dialog — 6 markers ready to print

2. **Place markers on the stage floor** — Position the printed markers at known locations on the stage:
 - **Minimum:** 3 markers (enough for a basic homography)
 - **Recommended:** 4-6 markers for better accuracy
 - **Placement strategy:**
 - Spread markers across the entire camera's field of view
 - Place at least one marker near each corner of the visible area
 - Place markers flat on the floor — tilted markers reduce accuracy
 - Measure each marker's position from the stage origin (back-right corner at floor level):

- X = distance from stage right (mm)
- Y = distance from back wall (mm)
- Z = 0 (floor level)
- Record the marker ID and its (X, Y) coordinates — you'll enter these in step 5

Part 2 — Register and Position the Camera

3. Register the camera — If the camera node is not already registered:

- Go to the Setup tab and click **Discover** — camera nodes respond to UDP broadcast
- Or go to Settings → Cameras → enter the camera's IP address manually
- Each USB camera sensor on the node appears as a separate fixture
- Verify the camera is online: its status should show "Online" with a green indicator



Camera configuration panel in Settings — camera list with IP, status, and calibration badges

Camera configuration panel in Settings — camera list with IP, status, and calibration badges

4. Position the camera in 3D — Switch to the Layout tab:

- Find the camera fixture in the sidebar (listed as "unplaced" if new)
- Drag it into the 3D viewport at the camera's real physical position
- Set the rotation to match the camera's actual aim direction:
 - A camera mounted on a wall at 2m height, aimed down at 30 degrees would have rotation Z=2000, tilt=-30
- In 3D view, the camera appears as a frustum (pyramid) showing its field of view
- Verify the frustum covers the area where you placed the ArUco markers

Part 3 — Run Calibration and Verify

5. Run ArUco calibration — In the Layout tab, click on the camera fixture to select it. Click the **Calibrate** button:

- The wizard opens and fetches a live snapshot from the camera
- The system automatically detects all visible ArUco markers and highlights them with green overlays
- **For each detected marker:**
 - The marker ID is shown on the overlay

- Enter the marker's real-world stage coordinates (X, Y in mm) that you measured in step 2
- Click **Record** to save the pixel-to-stage mapping for this marker
- After recording all markers, click **Compute** — the system builds a homography matrix that maps any pixel coordinate to stage floor coordinates



Camera snapshot with ArUco markers detected — green overlays showing marker IDs

Camera snapshot with ArUco markers detected — green overlays showing marker IDs

6. Review calibration results — The calibration summary shows:

- **Reprojection error:** How accurately the computed homography matches the recorded points. Lower is better:
 - < 10mm: Excellent — suitable for precision tracking
 - 10-20mm: Good — adequate for most use cases
 - 20-50mm: Fair — consider adding more markers or re-measuring positions
 - 50mm: Poor — re-check marker measurements and try again
- **Reference points:** Number of markers used (should match what you recorded)
- **Coverage area:** The stage area covered by the calibration (larger is better)



Calibration complete — reprojection error, reference points, and coverage summary

Calibration complete — reprojection error, reference points, and coverage summary

7. Save and apply — Click **Save** to persist the calibration:

- The camera fixture badge updates to show a green “Cal” checkmark
- All features that depend on pixel-to-stage conversion now use this calibration:
 - **Person tracking:** Detected bounding boxes are converted to stage positions
 - **Beam detection:** Beam centroids become stage coordinates for mover calibration
 - **Mover calibration:** The entire mover calibration wizard (Example C) requires this
- Calibration data is included in project file exports (.slyshow) for portability

Tips for accurate calibration: - **Marker size matters:** Use the 150mm markers at standard print size. Smaller markers are harder to detect at distance. - **Flat placement is critical:** Even a slight tilt (marker on a crumpled surface) can shift the detected center by 10-20mm. - **Cover the edges:** The homography is most accurate within the convex hull of your reference markers. Place markers at the extremes of the camera's view, not just the center. - **Lighting conditions:** ArUco detection works in most lighting, but avoid direct glare on the printed markers (glossy paper under bright lights). - **Re-calibrate when:** - The camera is physically moved (even slightly) - The camera lens is changed or zoom is adjusted - Stage dimensions change (markers would be at different positions) - Accuracy of tracking or beam detection degrades

Example E: Spotlight Follow Person — Live Tracking Preset (#382)

Use the built-in **Spotlight: Follow Person** preset to make moving heads automatically follow people detected by a camera in real-time.

Prerequisites: - At least one camera node online with person detection working (verify with a manual Scan first) - At least one DMX moving head fixture placed on the Layout tab - Camera calibration complete (see Example D) for accurate stage positioning - Mover calibration complete (see Example C) for accurate pan/tilt aiming - Art-Net/sACN engine running

Steps:

1. **Load the preset** — Go to the Runtime tab. Click **Load Show** (or the preset dropdown). Select **Spotlight: Follow Person** from the preset list.
 - If no camera node is registered, a warning appears: “No camera node registered — person detection will not work”
 - If no moving heads are configured, a warning appears about missing movers
 - The preset loads even with warnings — you can add the missing hardware later
2. **What it creates** — The preset automatically configures:
 - A **Track action** (type 18) on every available moving head, targeting `objectType: "person"`
 - A warm spotlight color (255, 240, 200) at full dimmer for the beam
 - A dim blue ambient wash (10, 5, 30) on all LED fixtures for atmospheric framing
 - A 10-minute looping timeline that keeps the DMX playback loop running

3. **Start camera tracking** — Click the **Track** button on the Setup tab or in the camera fixture edit modal. The camera node begins running detection using the configured tracking classes and parameters (see [Tracking Configuration](#)). Detected objects appear as labeled markers in the 3D viewport.
4. **Start playback** — Click **Bake**, then **Start**. The 40 Hz DMX playback loop begins. The Track action reads all temporal person objects and computes pan/tilt for each head in real-time.
5. **Walk on stage** — Within 2 seconds of entering the camera's view, a pink person marker appears. Moving heads light up with the warm spotlight color and aim at you. As you move, the beams follow.

Behavior with multiple people: - 1 person, 2 heads: Both heads aim at the same person - 2 people, 2 heads: One head per person (auto-spread) - 3+ people, 2 heads: Heads cycle through people every 2 seconds

When no one is detected: - Heads dim to 0 (blackout) and hold their last position - As soon as a person is detected again, heads immediately re-aim and light up

Tips: - Use a narrow beam profile (8-15 degrees) for a dramatic spotlight effect - Ensure the room is dim enough for the camera to distinguish the beam from ambient light - If tracking seems jittery, increase the camera's capture FPS or reduce the confidence threshold - The Track action works alongside other timeline effects — you can add spatial color washes on lower-priority tracks

19. API Quick Reference

Stage & Layout

METHOD	ENDPOINT	DESCRIPTION
GET/POST	<code>/api/layout</code>	Layout with fixtures and positions
GET/POST	<code>/api/stage</code>	Stage dimensions (w, h, d meters)
GET/POST	<code>/api/objects</code>	Stage objects (walls, floors, trusses, props)
POST	<code>/api/objects/temporal</code>	Create temporal objects (TTL-based)

Fixtures

METHOD	ENDPOINT	DESCRIPTION
GET/POST	<code>/api/fixtures</code>	List / create
GET/PUT/DELETE	<code>/api/fixtures/:id</code>	CRUD
PUT	<code>/api/fixtures/:id/aim</code>	Set aim point

Shows & Timelines

METHOD	ENDPOINT	DESCRIPTION
GET/POST	<code>/api/timelines</code>	List / create
POST	<code>/api/timelines/:id/bake</code>	Start baking
POST	<code>/api/timelines/:id/start</code>	Start playback
GET	<code>/api/show/presets</code>	List preset shows
GET/POST	<code>/api/show/export</code> , <code>/api/show/import</code>	Save/load show file

DMX

METHOD	ENDPOINT	DESCRIPTION
GET	<code>/api/dmx-profiles</code>	List profiles
GET	<code>/api/dmx/patch</code>	Universe address map
POST	<code>/api/dmx/start</code> , <code>/api/dmx/stop</code>	Engine control

Cameras

METHOD	ENDPOINT	DESCRIPTION
GET	<code>/api/cameras</code>	List registered camera fixtures
POST	<code>/api/cameras</code>	Register a camera node as fixture
DELETE	<code>/api/cameras/:id</code>	Remove camera fixture
GET	<code>/api/cameras/:id/snapshot</code>	Proxy JPEG snapshot
GET	<code>/api/cameras/:id/status</code>	Live status from camera node
POST	<code>/api/cameras/:id/scan</code>	Object detection (proxy to node <code>/scan</code>)
GET	<code>/api/cameras/discover</code>	Find camera nodes on network
GET/POST	<code>/api/cameras/ssh</code>	SSH credentials for deployment
POST	<code>/api/cameras/deploy</code>	Deploy firmware to camera node via SSH+SCP
GET	<code>/api/cameras/deploy/status</code>	Poll deploy progress

Camera Node Local API (port 5000)

METHOD	ENDPOINT	DESCRIPTION
GET	<code>/status</code>	Node status, capabilities, camera list
GET	<code>/config</code>	HTML config page with detection UI
GET	<code>/snapshot?cam=N</code>	JPEG snapshot from camera N
POST	<code>/scan</code>	Object detection (JSON: cam, threshold, resolution, classes)
GET	<code>/health</code>	Health check

20. Glossary

SlyLED touches lighting, networking, computer vision, and embedded firmware — which means a lot of acronyms and jargon. This section expands every acronym used elsewhere

in the manual and defines the domain terms that don't have a literal expansion ("baking", "universe", "blink-confirm", etc.).

Entries are alphabetised on the **Term** column. For acronyms that cluster around a common concept (e.g. RX / RY / RZ) the cluster appears under the first member.

TERM	EXPANSION	PLAIN-LANGUAGE DEFINITION	WHERE IT SHOWS UP
API	Application Programming Interface	The set of HTTP endpoints a program exposes for other programs to call.	§19 API Quick Reference; <code>/api/*</code> routes throughout.
ARM	Advanced RISC Machine	CPU architecture used by the Giga R1, Raspberry Pi, and Orange Pi. “Slow on ARM” in the manual means these boards.	§14 Camera Nodes (depth-estimation runtimes).
Art-Net	—	DMX-over-Ethernet protocol by Artistic Licence. The orchestrator sends <code>ArtDMX</code> packets to an Art-Net bridge, which relays them to DMX fixtures.	§2 Walkthrough Step 3b; §12 DMX Profiles.
ArtDMX	Art-Net DMX packet	One 512-channel Art-Net data packet.	§2 Walkthrough Step 3b.
ArtPoll	Art-Net discovery packet	The Art-Net discovery broadcast used to find bridges.	§2 Walkthrough Step 3a.
baking	—	Compiling a timeline into a pre-computed DMX scene stream so playback doesn’t need to recompute effects on every frame.	§10 Baking & Playback.
battleship search	—	Calibration-discovery strategy that probes a coarse grid across the full pan/tilt range before refining — faster than a dense scan when the beam’s reachable region is small.	Appendix B §B.3 Discovery.
BFS	Breadth-First Search	Graph-traversal algorithm that explores outward from a seed point one ring at a time. Used in mover calibration to map the visible-region boundary from a first detected beam position.	Appendix B §B.3 Mapping.
blink-confirm	—	Reflection-rejection check: after detecting a candidate beam pixel, nudge pan and	Appendix B §B.3; issue #658.

TERM	EXPANSION	PLAIN-LANGUAGE DEFINITION	WHERE IT SHOWS UP
		tilt slightly and verify the detected pixel actually moves. A reflection stays put; a real beam moves.	
CLI	Command-Line Interface	A text shell interface (e.g. <code>arduino-cli</code> , <code>gh</code> , PowerShell, bash). The orchestrator's build/flash flow is driven through CLI tools rather than IDEs.	§15 Firmware; CLAUDE.md build commands.
CPU	Central Processing Unit	The main processor. CPU0 / CPU1 distinguish the two cores on the ESP32 (WiFi pinned to one, FastLED pinned to the other).	§14 Camera Nodes; CLAUDE.md hardware quirks.
CRGB	Color RGB	FastLED's C++ struct for a single RGB pixel.	Firmware modules (<code>GigaLED.h</code>).
CRUD	Create, Read, Update, Delete	Shorthand for “all four basic database-style operations.”	§4 Fixture Setup; §12 DMX Profiles.
CSI	Camera Serial Interface	Raspberry Pi's native ribbon-cable camera port (not supported in SlyLED v1.x — use USB cameras).	§14 Camera Nodes.
CV	Computer Vision	Algorithmic image-understanding — the umbrella over solvePnP, RANSAC, beam detection, depth estimation, and YOLO. The auto-tune analyzer evaluator is the CV-only alternative to the AI evaluator.	Appendix A; §14 Camera Nodes; Appendix D.
dark reference	—	Snapshot captured with all calibration beams off, subtracted from subsequent frames so beam detection isn't fooled by ambient lighting.	Appendix A §A.5; issue #651.
DFU	Device Firmware Update	USB protocol for flashing microcontroller firmware in bootloader mode. The Giga R1 (USB ID <code>2341:0366</code>)	§15 Firmware; CLAUDE.md first-time setup.

TERM	EXPANSION	PLAIN-LANGUAGE DEFINITION	WHERE IT SHOWS UP
		requires the Zadig WinUSB driver installed once before DFU uploads work.	
DHCP	Dynamic Host Configuration Protocol	How devices on a network get an IP address. The board's hostname shows up in DHCP so routers list it by name.	§14 Camera Nodes deployment.
DMX	Digital Multiplex	Industry-standard lighting-control protocol — 512 channels per universe, carried over a twisted-pair cable or over Ethernet (Art-Net / sACN).	§2 Walkthrough; §12 DMX Profiles; Appendix B.
DNS	Domain Name System	The naming layer that resolves a hostname to an IP address. SlyLED relies on mDNS (zero-config DNS over multicast) for peer-name resolution on the LAN.	§14 Camera Nodes deployment.
DOF	Degrees of Freedom	Independent axes a system can move along. SlyLED's parametric mover model has 6 DOF (yaw, pitch, roll, pan offset, tilt offset, plus scale).	Appendix B §B.3 Model fit.
EEPROM	Electrically Erasable Programmable Read-Only Memory	Flash-backed non-volatile key/value storage on the D1 Mini (the ESP32 uses NVS instead). Each child board persists its hostname, string config, and per-string GPIO/length there.	§4 Fixture Setup; CLAUDE.md per-board limits.
ESP32	—	Espressif microcontroller family used for LED-performer nodes (WiFi + dual-core, up to 8 LED strings).	§4 Fixture Setup; §15 Firmware.
extrinsic	—	A camera's pose (position + rotation) in stage/world space. The solvePnP output. Pair with intrinsic .	Appendix A §A.4.
FastLED	—	Arduino library for driving WS2812B-style addressable	§15 Firmware; CLAUDE.md hardware quirks.

TERM	EXPANSION	PLAIN-LANGUAGE DEFINITION	WHERE IT SHOWS UP
		LED strips. Used on ESP32 and D1 Mini; not reliable on the Giga R1 (custom PWM path instead).	
fixture	—	Any addressable lighting device — an LED strip, a DMX wash, a moving head, or a camera (which registers as a placeable “fixture” so it has a layout position).	§4 Fixture Setup; throughout.
FOV	Field of View	The angular width a camera or lens sees. Stored as <code>fovDeg</code> + <code>fovType</code> (horizontal/vertical/diagonal). Used as an intrinsic fallback when true calibrated intrinsics aren’t available.	Appendix A §A.3.
FPS	Frames Per Second	Update rate for live playback or emulation.	§11 Show Preview Emulator.
FQBN	Fully Qualified Board Name	The arduino-cli identifier for a board target, e.g. <code>arduino:mbed_giga:giga</code> .	§15 Firmware.
GET / POST / PUT / DELETE	—	HTTP methods. GET reads, POST creates/triggers, PUT updates, DELETE removes.	§19 API Quick Reference.
GPIO	General-Purpose Input/Output	A configurable pin on a microcontroller — used for LED data lines on the ESP32.	§4 Fixture Setup (ESP32 only).
homography	—	A 3×3 matrix that maps points on one plane to points on another via projective transform. SlyLED uses a pixel↔floor homography as a fast alternative to full 3D extrinsics during calibration.	Appendix A §A.4.
HSV	Hue, Saturation, Value	A colour representation used for colour-filter beam detection (hue bands identify “the green beam” regardless of brightness).	Appendix A §A.8 Beam detection.

TERM	EXPANSION	PLAIN-LANGUAGE DEFINITION	WHERE IT SHOWS UP
HTML	HyperText Markup Language	The markup the SPA is built from.	§3 Platform Guide.
HTTP	HyperText Transfer Protocol	The request/response protocol behind every <code>/api/*</code> call and every config page served by the orchestrator, performers, and camera nodes.	§19 API Quick Reference; throughout.
HUD	Heads-Up Display	An overlay showing live state (used in the 3D viewport).	§5 Stage Layout.
ID	Identifier	Any short key that uniquely names something (fixture ID, ArUco marker ID, etc.).	Throughout.
IK	Inverse Kinematics	Given a target point, compute the pan/tilt values that aim the beam there. The parametric mover model provides IK once calibration completes.	Appendix B §B.3 Model fit.
intrinsic	—	A camera’s internal optical parameters: focal length (<code>fx</code> , <code>fy</code>), principal point (<code>cx</code> , <code>cy</code>), and lens distortion. Independent of where the camera is — that’s the extrinsic .	Appendix A §A.3.
IP	Internet Protocol	Addressing scheme for networked devices (<code>192.168.x.y</code>).	§14 Camera Nodes.
JPEG	Joint Photographic Experts Group	Compressed image format used for camera snapshots.	§14 Camera Nodes.
JSON	JavaScript Object Notation	The text format used for API request/response bodies and persisted data files.	§19 API Quick Reference.
kinematic model	—	Mathematical model that describes how a fixture’s motors translate pan/tilt DMX values into an aim direction in stage space. SlyLED fits a 6-	Appendix B §B.3.

TERM	EXPANSION	PLAIN-LANGUAGE DEFINITION	WHERE IT SHOWS UP
		DOF kinematic model per calibrated moving head.	
LAN	Local Area Network	The physical/WiFi network the orchestrator and performers share.	§2 Walkthrough.
LED	Light-Emitting Diode	Addressable RGB LEDs (WS2812B and similar) are the primary fixture type.	§4 Fixture Setup.
LM	Levenberg-Marquardt	Nonlinear least-squares solver used to fit the parametric mover model to calibration samples.	Appendix B §B.3 Model fit.
LSQ	Least-Squares	The fitting technique LM refines. When calibration falls back to “median-based” fitting, it’s because LSQ wouldn’t converge.	Appendix A §A.6.
MAC	Media Access Control	A hardware-unique 48-bit identifier baked into every WiFi/Ethernet interface. Performer hostnames are derived from the MAC (e.g. <code>SLYC-XXXX</code>).	§4 Fixture Setup; performer naming.
Mbed OS	—	The real-time operating system running on the Arduino Giga R1. Explains why <code>analogWrite()</code> and some libraries behave differently on the Giga.	CLAUDE.md hardware quirks.
mDNS	Multicast DNS	Zero-config DNS over multicast — how “SLYC-1234.local” resolves on the LAN without a DNS server.	§14 Camera Nodes deployment.
NTP	Network Time Protocol	How performers sync their clocks so runner start times are coordinated.	Protocol (<code>Globals.cpp</code>).

TERM	EXPANSION	PLAIN-LANGUAGE DEFINITION	WHERE IT SHOWS UP
NVS	Non-Volatile Storage	ESP32 flash-backed key/value store. SlyLED uses the <code>"slyled"</code> namespace. Equivalent to EEPROM on the D1 Mini.	§4 Fixture Setup.
ONNX	Open Neural Network Exchange	Portable neural-network file format. YOLOv8n and Depth-Anything-V2 ship as ONNX files so they run via <code>onnxruntime</code> on ARM.	§14 Camera Nodes.
OFL	Open Fixture Library	Community-maintained DMX-fixture profile database. SlyLED can import OFL JSON.	§12 DMX Profiles.
orchestrator	—	The desktop (Windows/Mac) or Giga-parent Flask server that hosts the SPA, designs shows, and drives performers and cameras. One of the three tiers.	§1 Getting Started.
OS	Operating System	—	CLAUDE.md hardware.
OTA	Over-the-Air	Firmware update pushed over WiFi instead of via USB.	§15 Firmware & OTA Updates.
PDF	Portable Document Format	The packaged manual format. Generated by <code>tests/build_manual.py</code> .	Appendix C.
performer	—	An ESP32, D1 Mini, or Giga-child LED execution node. One of the three tiers.	§1 Getting Started.
PnP / solvePnP	Perspective-n-Point	OpenCV algorithm that computes a camera's 3D pose from ≥ 3 known 2D↔3D point correspondences. <code>SOLVEPNP_SQPNP</code> is the preferred solver; <code>SOLVEPNP_ITERATIVE</code> is the fallback.	Appendix A §A.4.
PNG	Portable Network Graphics	Lossless image format used for screenshots.	§2 Walkthrough.

TERM	EXPANSION	PLAIN-LANGUAGE DEFINITION	WHERE IT SHOWS UP
PR	Pull Request	Git/GitHub workflow — a proposed change on a branch, reviewed before merge.	Appendix C §C.4.
PWM	Pulse-Width Modulation	Dimming technique where the LED is switched on and off fast. On the Giga R1 this is implemented in software because <code>analogWrite()</code> is banned on the onboard RGB pins.	CLAUDE.md hardware quirks.
QA	Quality Assurance	Testing role — in SlyLED’s workflow, QA runs the Playwright + test suites and files issues rather than patching source.	Appendix C.
QR	Quick Response (code)	2D barcode. Not the same as an ArUco marker — ArUco is designed for solvePnP, QR for data payloads.	—
RANSAC	Random Sample Consensus	Robust plane-fitting algorithm — samples random small subsets, finds the model with the most inliers. SlyLED uses it to detect floor and wall planes in noisy point clouds.	Appendix A §A.7.
reprojection RMS	—	After solvePnP, project the 3D points back through the solved pose and measure the pixel distance to the detected corners. Reported as root-mean-square across all points. <2 px is excellent, 2–5 px is usable, >5 px means something is wrong.	Appendix A §A.4.
REST	Representational State Transfer	An architectural style for HTTP APIs — predictable verbs (GET/POST/PUT/DELETE) acting on noun-shaped URLs. SlyLED’s <code>/api/*</code> surface follows REST conventions.	§19 API Quick Reference.

TERM	EXPANSION	PLAIN-LANGUAGE DEFINITION	WHERE IT SHOWS UP
RGB / RGBW	Red, Green, Blue [, White]	Standard LED colour models. RGBW adds a dedicated white LED for purer whites.	§4 Fixture Setup.
RMS	Root-Mean-Square	Quadratic-mean aggregation of errors (<code>sqrt(mean(x²))</code>). More sensitive to outliers than a plain mean — which is why it's used as a calibration quality metric.	Appendix A §A.4.
Rodrigues	—	Mathematical conversion between a rotation vector (<code>rvec</code> from <code>solvePnP</code>) and a 3×3 rotation matrix. <code>cv2.Rodrigues()</code> .	Appendix A §A.4.
RSSI	Received Signal Strength Indicator	How strong a WiFi signal a performer is hearing. Reported in dBm; the orchestrator stores it as an unsigned magnitude so “69” means “−69 dBm”.	UDP protocol PONG payload.
RTOS	Real-Time Operating System	An OS with deterministic timing guarantees. Mbed OS on the Giga is an RTOS.	CLAUDE.md hardware.
runner	—	A step sequencer loaded into a performer. Each step is an action (colour, pattern, LED range) with a duration; the runner loops the step list in sync with the orchestrator.	§4 Fixture Setup; §13 Preset Shows.
RX / RY / RZ	—	Rotations about the X, Y, Z axes of the stage frame, in degrees. In schema v2: <code>rx</code> = pitch, <code>ry</code> = roll, <code>rz</code> = yaw/pan. Never read <code>rotation[1]</code> or <code>rotation[2]</code> directly — always go through <code>rotation_from_layout()</code> .	Appendix A §A.9.
sACN	Streaming ACN	DMX-over-Ethernet alternative to Art-Net, defined by RFC 7724. SlyLED speaks both.	§12 DMX Profiles.

TERM	EXPANSION	PLAIN-LANGUAGE DEFINITION	WHERE IT SHOWS UP
SCP	Secure Copy Protocol	File transfer over SSH. How camera firmware reaches the Orange Pi / Raspberry Pi.	§15 Firmware → Camera deploy.
solvePnP	—	See PnP .	Appendix A §A.4.
SPA	Single-Page Application	The desktop orchestrator UI is one HTML page that loads JavaScript modules instead of navigating between pages.	§3 Platform Guide.
SQPNP	—	A specific solvePnP algorithm variant (<code>cv2.SOLVEPNP_SQPNP</code>) chosen because it tolerates fewer correspondences than the iterative solver.	Appendix A §A.4.
SRAM	Static Random-Access Memory	The fast, volatile RAM on a microcontroller. Tight budget on the D1 Mini — the manual warns against String objects and heap allocation.	CLAUDE.md performance rules.
SSH	Secure Shell	Encrypted remote-login protocol. How the orchestrator reaches camera-node shells for firmware deployment.	§15 Firmware → Camera deploy.
SVG	Scalable Vector Graphics	Vector image format used by diagram exporters.	Appendix C.
TCP	Transmission Control Protocol	Reliable, connection-based networking. HTTP traffic (config pages, API calls) rides on TCP.	UDP protocol discussion.
tiling	—	Sliced-Aided Hyper-Inference (SAHI)-style detection: break a large image into overlapping patches, run the detector on each, stitch results. Improves small-object detection accuracy at the cost of runtime. Controlled by the <code>tile</code> option on <code>/scan</code> .	§14 Camera Nodes.
TTL	Time-To-Live	A timeout after which a resource (e.g. a mover claim)	Appendix B §B.7.

TERM	EXPANSION	PLAIN-LANGUAGE DEFINITION	WHERE IT SHOWS UP
		auto-expires. Mover-control claims have a 15 s TTL.	
UDP	User Datagram Protocol	Connectionless, best-effort networking. Used for all orchestrator↔performer traffic (discovery, actions, runner control) because it's low-latency and the wire protocol tolerates occasional packet loss.	Wire protocol; CLAUDE.md §UDP binary protocol.
UI	User Interface	—	Throughout.
universe	—	A DMX addressing space — 1–512 channels. A show typically spans multiple universes; Art-Net addresses them as <code>net.subnet.universe</code> .	§12 DMX Profiles.
URL	Uniform Resource Locator	Web address.	§14 Camera Nodes.
USB	Universal Serial Bus	—	§15 Firmware USB flash.
V4L2	Video for Linux 2	The kernel video-capture API used by camera nodes (<code>cv2.VideoCapture</code> on Orange Pi / Raspberry Pi). SoC ISP video nodes like <code>sunxi-vin</code> and <code>bcm2835-isp</code> are filtered out — only regular USB cameras register.	§14 Camera Nodes.
WiFi	—	802.11 wireless networking. Performers and camera nodes join the orchestrator's LAN over WiFi.	§2 Walkthrough.
WLED	—	Popular open-source firmware for ESP32/8266-based LED controllers. SlyLED includes a bridge so WLED devices can appear as performers.	§4 Fixture Setup; <code>desktop/shared/wled_bridge.py</code> .
WS2812B	—	Common addressable-RGB LED chip (aka “NeoPixel”).	§4 Fixture Setup.

TERM	EXPANSION	PLAIN-LANGUAGE DEFINITION	WHERE IT SHOWS UP
		The ESP32 RMT peripheral drives it in hardware; the D1 Mini bit-bangs it in software.	
YOLO	You Only Look Once	Single-pass object-detection neural network. SlyLED camera nodes run YOLOv8n via ONNX Runtime for person/object detection on <code>POST /scan</code> .	§14 Camera Nodes.
ZIP	—	Archive file format, used for the release bundle.	§15 Firmware Registry.

Not sure what something means? If a term appears in the manual but isn't in this table, that's a bug in the glossary — open an issue or PR against [#663](#).

Appendix A — Camera Calibration Pipeline (DRAFT)

⚠ **DRAFT — assumes all in-flight work is merged.** This appendix describes the camera-calibration pipeline under the assumption that issues [#610](#), [#651–#661](#), and [#357](#) are fully implemented. Some features documented below are **partially merged** today (notably full intrinsic calibration of every camera, dark-reference integration into the mover pipeline per [#651](#), and the floor-view polygon target filter per [#659](#)). See `docs/DOCS_MAINTENANCE.md` for the current merge status and the criteria for removing this banner. Issue [#662](#).

Camera calibration runs as a one-time setup per camera node and must be repeated whenever a camera is physically moved or re-aimed. It produces, per camera: an **intrinsic** matrix **K** (focal length + principal point + distortion), an **extrinsic** pose (stage-space position + rotation), and — for stages that will run point-cloud scans — a depth-anchor fit that corrects monocular depth to stage metric.

A.1 Pipeline overview

```
%%{init: {'theme':'neutral'}}%%
flowchart TD
    Start([Register camera node]) --> Deploy[Deploy firmware via SSH+SCP]
    Deploy --> Intrinsic{1 – Intrinsic calibration}
    Intrinsic -->|Checkerboard path| CBCap[Capture 15-30 checkerboard frames<br/>~500ms/frame]
    Intrinsic -->|ArUco path| ArCap[Capture 5-10 ArUco snapshots]
    CBCap --> CBCompute[cv2.calibrateCamera<br/>~2s for 3 frames]
    ArCap --> ArCompute[cv2.calibrateCamera<br/>a few seconds]
    CBCompute --> Saved[Save intrinsic_camN.json on camera]
    ArCompute --> Saved
    Intrinsic -->|skip| FOV[Fall back to FOV-derived K<br/>intrinsic_source = fov-estimate]

    Saved --> Survey[2 – ArUco marker survey]
    FOV --> Survey
    Survey --> MarkerReg[POST /api/aruco/markers<br/>id, size, x/y/z, rx/ry/rz, label]
    MarkerReg --> Coverage[GET /api/aruco/markers/coverage]
    Coverage --> Enough{>= 3 markers<br/>visible per camera?}
    Enough -->|no| AddMarker[Add marker at<br/>recommendation pin]
    AddMarker --> Coverage
    Enough -->|yes| DarkRef[3 – Dark-reference capture<br/>~100ms/cam]

    DarkRef --> StageMap[4 – Extrinsic solve<br/>~5-10s multi-snapshot]
    StageMap --> PnP[cv2.solvePnP SQPNP<br/>best-per-ID corners]
    PnP --> StoreExt[Store rotation rx/ry/rz + position<br/>rotationSchemaVersion: 2]

    StoreExt --> Scan[5 – Optional: space scan]
    Scan --> PointCloud[/point-cloud per cam<br/>~6.5s/cam metric model]
    PointCloud --> Anchor[Depth anchor fit<br/>scale + offset]
    Anchor --> Merge[space_mapper<br/>cross-cam filter + floor normalize]
    Merge --> Surfaces[surface_analyzer RANSAC<br/>floor + walls + obstacles]
    Surfaces --> Done([Stage geometry ready])

    Scan -->|skip| Done
    StageMap -->|RMS > 5px or too few markers| Error[Calibration error]
    PnP -->|no convergence| Error
    Error --> Retry[Add markers or increase snapshots]
    Retry --> Coverage
```

A.2 ArUco marker surveying

Physical ArUco markers mounted on the stage floor, walls, or rigging define the ground-truth geometry that every subsequent camera-calibration step references.

Registry schema — each marker is stored with:

FIELD	TYPE	NOTES
<code>id</code>	int (0–49)	ID within the <code>CV2_DICT_4X4_50</code> dictionary
<code>size</code>	float mm (≥ 1)	Physical edge length; default 100 mm
<code>x</code> , <code>y</code> , <code>z</code>	float mm	Stage-space position; usually <code>z=0</code> for floor markers
<code>rx</code> , <code>ry</code> , <code>rz</code>	float deg	Marker orientation — see §A.9 for axis convention
<code>label</code>	string (≤ 60 chars)	Operator annotation, e.g. <code>north-entrance</code>

Endpoints

METHOD	PATH	PURPOSE
GET	<code>/api/aruco/markers</code>	Full registry (<code>dictId</code> , <code>markers[]</code>)
POST	<code>/api/aruco/markers</code>	Upsert one or more markers by ID
DELETE	<code>/api/aruco/markers/<id></code>	Remove a marker by ID
GET	<code>/api/aruco/markers/coverage</code>	Per-camera visible-ID report with a placement recommendation

Coverage pre-flight — before starting the extrinsic solve, run `GET /api/aruco/markers/coverage`. The response lists which IDs each camera currently sees, the stage-space hull covered by the registered markers, and a `recommendation` object indicating which camera has the weakest coverage and where the operator should place the next marker.

Expected timing — marker registration is instant (JSON write). Coverage check takes one snapshot per registered camera, typically 50–200 ms per camera.

Fallback — none. Without surveyed markers there is no extrinsic solve; the camera defaults to identity rotation and `(0, 0, 0)` position.

A.3 Intrinsic calibration

Intrinsic calibration produces the camera’s **K** matrix (`fx`, `fy`, `cx`, `cy`) and distortion coefficients. It is a per-lens, per-resolution, one-time step. Two independent paths are supported:

Path A — Checkerboard (camera-side)

METHOD	PATH	PURPOSE
POST	<code>/calibrate/intrinsic/capture</code>	Grab one frame, find ≤ 10 checkerboards (4×9, 25 mm squares default), accumulate corners
POST	<code>/calibrate/intrinsic/compute</code>	Run <code>cv2.calibrateCamera</code> on accumulated frames; save to <code>/opt/slyled/calib/intrinsic_camN.json</code>
GET	<code>/calibrate/intrinsic?cam=N</code>	Retrieve the saved calibration
DELETE	<code>/calibrate/intrinsic</code>	Remove saved calibration
POST	<code>/calibrate/intrinsic/reset</code>	Clear accumulated frames (does not touch saved file)

Expected timing — ~500 ms per frame capture, 2-5 s compute with the minimum three frames. Targets for a usable calibration: 15-30 frames, RMS < 0.3 pixels.

Path B — ArUco (orchestrator-side)

METHOD	PATH	PURPOSE
POST	<code>/api/cameras/<fid>/aruco/capture</code>	Snapshot + ArUco detection, accumulate per-fixtured corners
POST	<code>/api/cameras/<fid>/aruco/compute</code>	Pool corners across frames, <code>cv2.calibrateCamera</code> , POST result to camera node for persistence
POST	<code>/api/cameras/<fid>/aruco/reset</code>	Clear accumulated frames
GET	<code>/api/cameras/<fid>/intrinsic</code>	Proxy GET to the camera node
DELETE	<code>/api/cameras/<fid>/intrinsic</code>	Proxy DELETE to the camera node
POST	<code>/api/cameras/<fid>/intrinsic/reset</code>	Proxy POST to the camera node

Expected timing — 5-10 captures × a few hundred ms each, compute in a few seconds.

Fallback — if no calibration file is available, the orchestrator falls back to an FOV-derived K on the fly: $fx = (w/2) / \tan(h_{fov}/2)$, $fy = fx$, $cx = w/2$, $cy = h/2$, distortion zero. The `/stage-map` response reports `intrinsic_source: "fov-estimate"` when this path is used. Accuracy drops to roughly $\pm 15\%$ of true focal length.

Persistence — saved to the camera node at `/opt/slyled/calib/intrinsic_camN.json`; survives reboots.

A.4 Extrinsic solve (solvePnP)

Given surveyed markers in stage space and their detected pixel corners, the orchestrator computes each camera's pose via `cv2.solvePnP` (see [PnP](#), [RANSAC](#) is **not** used here — PnP is a direct algebraic solve, not a consensus method).

```
%%{init: {'theme':'neutral'}}%%
sequenceDiagram
    actor Op as Operator
    participant SPA as SPA (calibration.js)
    participant Orch as Orchestrator
    participant Cam as Camera node

    Op->>SPA: Click Solve
    SPA->>Orch: POST /api/cameras/fid/stage-map

    Note over Orch,Cam: Multi-snapshot aggregation
    loop maxSnapshots (default 6)
        Orch->>Cam: GET /snapshot?cam=N
        Cam-->>Orch: JPEG
        Orch->>Orch: detectMarkers + keep best per ID
    end

    Orch->>Cam: GET /calibrate/intrinsic?cam=N
    Cam-->>Orch: K or 404
    alt K available
        Orch->>Orch: intrinsic_source=calibrated
    else fallback
        Orch->>Orch: intrinsic_source=fov-estimate
    end

    Orch->>Orch: cv2.solvePnP SQPNP
    alt convergence
        Orch->>Orch: projectPoints RMS check
        Orch->>Orch: Rodrigues -> tilt/pan/roll
        Orch-->>SPA: ok, reprojectionRmsPx
    else RMS > 5 or no convergence
        Orch->>Orch: try SOLVEPNP_ITERATIVE
        alt still fails
            Orch-->>SPA: error
        end
    end
end
```

Endpoint — `POST /api/cameras/<fid>/stage-map` with `{cam, markers, markerSize, maxSnapshots}`.

Preconditions — ≥ 3 surveyed markers registered (or ≥ 2 if all floor-coplanar), each visible in at least one snapshot. Multi-snapshot aggregation handles frame-to-frame transients; six snapshots is a good default.

Algorithm — for each snapshot, the orchestrator detects markers and keeps the single detection with the largest perimeter per ID (largest = closest = best sub-pixel corners). Correspondences from each detected corner to the surveyed 3D point feed into `cv2.solvePnP(..., flags=SOLVEPNP_SQPNP)`, with `SOLVEPNP_ITERATIVE` as a fallback solver.

Expected timing — one multi-snapshot run: 5–10 s (dominated by snapshot capture).

Output — `tvec` (stage mm) → camera position; `rvec` → Rodrigues → tilt/pan/roll → stored in `camera.rotation` using schema v2 (§A.9). Also reports `reprojectionRmsPx`.

Fallbacks

FAILURE	BEHAVIOUR	OPERATOR ACTION
Fewer than 3 markers matched across all snapshots	Error; pose not updated	Add markers or reposition camera
SQPNP does not converge	Retry with <code>SOLVEPNP_ITERATIVE</code>	None automatic
Reprojection RMS > 5 px	Pose stored but flagged	Verify surveyed marker positions and <code>markerSize</code> ; recapture intrinsics
No intrinsics on camera	Fall back to FOV-derived K, report <code>intrinsic_source: "fov-estimate"</code>	Run §A.3 for better accuracy

A.5 Dark-reference capture (#651)

A dark-reference frame is a snapshot taken with all calibration beams off so that beam detection in subsequent calibration steps can subtract ambient lighting.

Endpoint — `POST /dark-reference` on the camera node, body `{cam: -1}` (all cameras) or `{cam: N}`.

Behaviour — captures one frame per camera, stores in the `BeamDetector` in-memory buffer (not persisted across reboots).

Expected timing — ~100 ms per camera (V4L2 frame grab).

When it runs — automatically at the start of each mover calibration run (see Appendix B §B.3). Can also be triggered manually before running beam-detect calls.

Fallback — if the beam detector module is not available on the node, the endpoint returns 503 and the caller proceeds without dark-reference subtraction. Beam detection still works but is more sensitive to ambient light.

A.6 Point cloud + multi-camera merge

Point-cloud generation produces a 3D representation of the stage from monocular depth plus camera pose. It is optional — only needed for features that reason about stage surfaces (mover-calibration target filtering, tracking, spatial effects on arbitrary geometry).

Camera-side endpoint — `POST /point-cloud` with `{cam, maxPoints, maxDepthMm}`. Returns `{points: [[x,y,z,r,g,b], ...], pointCount, inferenceMs, calibrated, fovDeg}`.

Depth models

MODEL	FILE	OUTPUT	TYPICAL INFERENCE ON ARM
Metric (Depth-Anything-V2 Metric Indoor Small)	<code>/opt/syled/models/depth_anything_v2_metric_indoor.onnx</code>	Depth in mm directly	~6.5 s / frame
Disparity (Depth-Anything-V2 Small, fallback)	<code>/opt/syled/models/depth_anything_v2_small.onnx</code>	Normalized [0,1]; caller scales by <code>maxDepthMm</code>	~6.5 s / frame

The camera selects the metric model by default; the disparity model is a fallback when the metric file is absent. Preference file: `/opt/syled/models/active_depth_model`.

Multi-camera merge — `POST /api/space/scan` on the orchestrator runs, in order:

1. Fetch per-camera point clouds (try `/point-cloud` on the node; orchestrator-side depth if camera path unavailable).
2. Per-camera depth anchor (#581) — two-parameter `scale + offset` least-squares fit so monocular depth agrees with stage geometry. Reject outliers $>2\sigma$, refit. If RMS > 2000 mm, fall back to a median-based coarse fit.
3. Transform to stage coordinates using the camera's stored rotation + position.

4. Cross-camera consistency filter (#582) — if ≥ 2 cameras are present, reject points that appear in only one camera's view where another camera should have seen them (filters monocular hallucinations).
5. Floor normalization — compute 5th-percentile Z per camera, shift cloud so the average floor lands at $Z=0$; fall back to RANSAC floor detection if the camera-anchored method fails.
6. Z-marker alignment (#599) — if any registered marker has `z < 50 mm` and zero rotation, treat it as ground truth and shift the cloud so those markers sit at $Z=0$.

Expected timing — 30-60 s end-to-end for a typical two-camera setup; dominated by per-camera depth inference.

Anchor quality classification in the response (`depthAnchor.quality`): `"ok"` ($\text{RMS} \leq 500 \text{ mm}$), `"degraded"` (500–2000 mm), `"fallback"` ($>2000 \text{ mm}$, median fit used). Degraded or fallback quality means downstream features (surface fitting, tracking) may be inaccurate.

A.7 Surface analysis (RANSAC)

After a merged point cloud is available, `desktop/shared/surface_analyzer.py` extracts structural surfaces so the orchestrator can reason about them (obstacle-aware target picking, beam-surface intersection for mover calibration).

SURFACE	ALGORITHM	EXPECTED RUNTIME	FAILURE MODE
Floor	RANSAC up to 200 trials, sample 3 points, plane fit, require vertical normal ($\text{dot-z} > 0.95$), $\geq 5\%$ inlier share	100–500 ms	Too few inliers → no floor reported
Walls	RANSAC on non-floor points, 2-point vertical-plane fit, ≥ 50 inliers or $\geq 5\%$, max 4 walls	500 ms–2 s total	Silent; fewer walls reported
Obstacles	300 mm XY grid + flood-fill, cluster size ≥ 20 points, classify as <code>pillar</code> if tall+thin else <code>obstacle</code>	100–500 ms	Silent; sparse clusters rejected

Ray-cast intersection — `beam_surface_check()` answers “which surface does a beam ray hit first?” Used by mover calibration to interpret detected-beam pixels as stage points when the beam lands on a wall or pillar rather than the floor (see #585/#260). Typical runtime 10–50 ms.

A.8 Beam detection (interface used by mover calibration)

Beam detection is documented here because it is *how the camera pipeline feeds moving-head calibration* (Appendix B). The endpoints live on the camera node.

METHOD	PATH	PURPOSE	TYPICAL RUNTIME
POST	<code>/beam-detect</code>	Single-frame detection; color filter + brightness + saturation + compactness	<100 ms
POST	<code>/beam-detect/flash</code>	Capture ON frame → wait <code>offDelayMs</code> → capture OFF → diff; immune to ambient shifts	<100 ms + <code>offDelayMs</code>
POST	<code>/beam-detect/center</code>	Multi-beam fixture: detect N beams, return cluster centre	<150 ms

Color filtering uses HSV hue ranges (`beam_detector.py`): red `[0,60] u [168,180]`, green `[35,85]`, blue `[100,130]`, magenta `[140,170]`; white falls back to brightness only.

Validation checks per contour: mean-V ≥ 160 (brightness); mean-S ≥ 80 for colored beams (saturation); aspect ≤ 5 (compactness).

A.9 Rotation schema v2 (axis convention)

Camera and DMX fixture rotations share a unified convention (issues #586, #600).

`fixture.rotation` is a list `[rx, ry, rz]` in **degrees**, axis-letter-matched to the Z-up stage frame:

```

%%{init: {'theme':'neutral'}}%%
flowchart LR
    subgraph Stage["Stage coordinate frame (Z-up)"]
        X["+X – stage-left"]
        Y["+Y – downstage forward"]
        Z["+Z – up"]
    end

    subgraph Rotation["rotation = [rx, ry, rz] (deg)"]
        RX["rx = pitch<br/>about X<br/>rx > 0 aims DOWN"]
        RY["ry = roll<br/>about Y (stage-forward)<br/>ry > 0 rotates image clockwise<br/>(viewed from"]
        RZ["rz = yaw / pan<br/>about Z (stage-up)<br/>rz > 0 aims toward +X"]
    end

    Stage --> Rotation

```

Canonical read path — always route through:

- Python: `desktop/shared/camera_math.py::rotation_from_layout(rot) → (tilt, pan, roll)`, then `build_camera_to_stage(tilt, pan, roll)` for the 3×3 matrix.
- SPA: `rotationFromLayout(rot)` in `spa/js/app.js`.

Never read `rotation[1]` **or** `rotation[2]` **directly** — those indices swap between v1 and v2 files.

Schema migration — imported project files carrying `layout.rotationSchemaVersion < 2` (or missing) have `ry` and `rz` swapped on load (v1 used `ry=pan, rz=roll`; v2 uses `ry=roll, rz=pan`). Current exports always write `rotationSchemaVersion: 2`.

A.10 Failure modes & operator expectations

PHASE	SYMPTOM	PROBABLE CAUSE	OPERATOR ACTION
Marker survey	Coverage recommendation persists after adding markers	Marker outside every camera's FOV	Move marker toward the recommended pin, or reposition camera
Intrinsic capture	"Board not found"	Lighting too low, angle too oblique, printout wavy	Flatten print, reposition, add light
Intrinsic compute	RMS > 1.0 px	Too few frames, poor angle variety	Capture 10+ more frames from diverse angles
Extrinsic solve	<code>markersMatched < 3</code>	Markers not visible in any snapshot	Increase <code>maxSnapshots</code> , reposition camera, add markers
Extrinsic solve	<code>reprojectionRmsPx > 5</code>	Bad intrinsics or wrong <code>markerSize</code>	Run §A.3; verify physical marker size matches registry
Extrinsic solve	<code>intrinsic_source: fov-estimate</code>	Camera has no saved intrinsic calibration	Optional: run §A.3 for $\pm 2\text{--}5$ px accuracy instead of $\pm 10\text{--}20$ px
Dark reference	503 response	Beam detector module missing on camera	Redeploy firmware from the Firmware tab
Depth estimate	"model unavailable"	ONNX file not deployed	Deploy via Firmware → Camera tab
Anchor fit	<code>quality: fallback</code>	Monocular depth disagrees strongly with stage geometry	Verify stage bounds and camera pose; may be a solvePnP error upstream
Surface analysis	No floor reported	Cloud too sparse or noisy, camera aimed too high	Add cameras, aim lower, verify depth model

A.11 File locations & persistence

DATA	LOCATION	FORMAT	NOTES
ArUco registry	<code>desktop/shared/data/aruco_markers.json</code>	JSON list	Persisted on every marker POST/DELETE
Camera fixtures	<code>desktop/shared/data/fixtures.json</code>	JSON list	Includes stored pose + rotation
Layout positions	<code>desktop/shared/data/layout.json</code>	JSON	Must carry <code>rotationSchemaVersion: 2</code>
Intrinsic calibration (camera-side)	<code>/opt/slyled/calib/intrinsic_camN.json</code>	JSON	<code>fx, fy, cx, cy, distCoeffs, imageSize, rmsError, frameCount</code>
Point cloud cache	<code>desktop/shared/data/pointcloud.json</code>	JSON	From last <code>/api/space/scan</code> run

Appendix B — Moving-Head Calibration Pipeline

Updated for orchestrator v1.7.83. The pipeline described here is the post-#784 architecture: a single canonical aim-vector (`aim_stage`, #806) is the source of truth for every head, the `ParametricFixtureModel` + Levenberg-Marquardt fit (#488) is the primary IK, and the legacy SMART pipeline that powered earlier calibration releases has been retired. The phase-by-phase content below has been ground-truthed against the current `mover_calibrator.py` and `parametric_mover.py`.

Moving-head calibration runs per [DMX](#) moving-head fixture after the camera(s) covering its reachable region have been calibrated (Appendix A). It produces a sample set + parametric 6-DOF [kinematic model](#) that lets the orchestrator translate stage-space targets into exact pan/tilt DMX values, enabling [IK](#) (inverse kinematics) for the Track action and spatial effects.

B.1 Pipeline overview

```
%%{init: {'theme':'neutral'}}%%
flowchart TD
    Start([Start Calibration]) --> Claim[Claim mover<br/>15s TTL]
    Claim --> Warmup[1 – Warmup<br/>~30s]
    Warmup --> Discovery{2 – Discovery<br/>45-55s}
    Discovery -->|Battleship path| BS[Battleship 4x4 grid<br/>+ blink-confirm nudge<br/>~10-15s]
    Discovery -->|Legacy path| Coarse[Coarse 10x7 scan<br/>+ fine spiral fallback<br/>~45-55s]
    BS --> MapConv[3 – Mapping / Convergence<br/>35-50s]
    Coarse --> MapConv
    MapConv --> Grid[4 – Grid build<br/>&lt;1s]
    Grid --> Verify[5 – Verification sweep<br/>3-5s advisory]
    Verify --> Fit[6 – LM model fit<br/>verify_signs → single-combo LM<br/>&lt;1s]
    Fit --> HoldOut{7 – Held-out<br/>parametric gate<br/>#654}
    HoldOut -->|pass| Save[8 – Save +<br/>moverCalibrated=true]
    HoldOut -->|fail| OperatorRetry[Accept / Retry prompt]
    OperatorRetry -->|Retry| Discovery
    OperatorRetry -->|Accept| Save
    Save --> Release[Release claim + blackout]
    Release --> Done([Complete])

    Discovery -.->|no beam after 80 probes| Error
    MapConv -.->|fewer than 6 samples| Error
    Fit -.->|no sign combo fits| Error
    Error[Phase error] --> Blackout[_cal_blackout<br/>512 zeros + release claim]
```

B.2 Phase reference

#	PHASE	STATUS STRING	TYPICAL DURATION	PROGRESS %	FALLBACK ON FAILURE
1	Warmup	warmup	30 s (configurable via warmupSeconds)	2-8	Log warning, continue without warmup
2	Discovery (legacy)	discovery	45-55 s	10-30	Abort with error status after 80 probes
2'	Discovery (battleship)	battleship → confirming	10-15 s	10-25	Abort with error ; can fall back to legacy discovery
3	Mapping (legacy BFS)	mapping	35-50 s	35-70	Abort if <6 samples
3'	Convergence (v2)	sampling	30-60 s (N targets × ~1 s each)	30-70	Abort if convergence fails on multiple targets
4	Grid build	grid	<1 s	~80	Abort if sample spread insufficient
5	Verification sweep	verification	3-5 s (3 held-out points)	~90	Advisory only — does not block save
6	Model fit (LM)	fitting	<1 s	85-95	verify_signs first → single-combo LM; full four-combo fallback if a sign probe misses
7	Held-out parametric gate (#654)	holdout	2-5 s (N unseen targets)	95-98	Surface Accept/Retry prompt
8	Save	complete	<1 s	100	Write error logged but does not affect moverCalibrated flag

Additional top-level statuses: cancelled , error , done .

```
%%{init: {'theme':'neutral'}}%%
```

```
stateDiagram-v2
```

```
    [*] --> Idle
```

```
    Idle --> Claimed: POST /claim
```

```
    Claimed --> Warmup: start
```

```
    Warmup --> Discovery
```

```
    Discovery --> Confirming: candidate found<br/>(battleship path)
```

```
    Confirming --> Discovery: nudge fails<br/>(#658 reflection)
```

```
    Confirming --> Mapping: nudge confirms
```

```
    Discovery --> Mapping: beam found<br/>(legacy path)
```

```
    Mapping --> Grid: samples >= 6
```

```
    Grid --> Verification
```

```
    Verification --> Fitting: advisory pass or skip
```

```
    Fitting --> HeldOut: LM converged
```

```
    HeldOut --> Complete: within tolerance
```

```
    HeldOut --> Idle: retry (operator)
```

```
    Complete --> Idle: released
```

```
    Discovery --> Error: 80 probes exhausted
```

```
    Mapping --> Error: insufficient spread
```

```
    Fitting --> Error: no sign combo fits
```

```
    Error --> Blackout
```

```
    Warmup --> Cancelled: operator cancel
```

```
    Discovery --> Cancelled: operator cancel
```

```
    Mapping --> Cancelled: operator cancel
```

```
    Confirming --> Cancelled: operator cancel
```

```
    Cancelled --> Blackout
```

```
    Warmup --> TimedOut: phase budget #653
```

```
    Discovery --> TimedOut: phase budget #653
```

```
    Mapping --> TimedOut: phase budget #653
```

```
    TimedOut --> Blackout
```

```
    Blackout --> Idle: claim released
```


B.3 Phase-by-phase detail

```
%%{init: {'theme':'neutral'}}%%
sequenceDiagram
    actor Op as Operator
    participant SPA as SPA (calibration.js)
    participant Orch as Orchestrator<br/>(mover_calibrator.py)
    participant DMX as Art-Net bridge
    participant Cam as Camera node

    Op->>SPA: Click Start Calibration
    SPA->>Orch: POST /api/calibration/mover/fid/start
    Orch->>Orch: _set_calibrating(fid, True)

    Note over Orch,DMX: Warmup (~30s)
    loop warmup sweep steps
        Orch->>DMX: _hold_dmx(pan,tilt,colour)
    end

    Note over Orch,Cam: Discovery (~45-55s or ~10-15s battleship)
    loop battleship grid or coarse 10x7
        Orch->>DMX: set pan,tilt
        Orch->>Cam: POST /beam-detect
        Cam-->>Orch: pixel or null
    end

    opt battleship hit – blink-confirm (#658)
        Orch->>DMX: pan + nudge
        Orch->>Cam: /beam-detect
        Orch->>DMX: tilt + nudge
        Orch->>Cam: /beam-detect
    end

    Note over Orch,Cam: Mapping / Convergence (~35-50s)
    loop BFS or per-target converge
        Orch->>DMX: set pan,tilt
        Orch->>Cam: /beam-detect (dual-capture #655)
    end

    Note over Orch: Grid build (<1s, sync)
    Note over Orch,Cam: Verification sweep (3-5s)
    Note over Orch: verify_signs probes → single-combo LM fit
    Note over Orch,Cam: Held-out parametric gate (#654)

    alt within tolerance
        Orch->>Orch: fixture.moverCalibrated=true
    else too high
        Orch-->>SPA: prompt Accept/Retry
    end

    Orch->>DMX: blackout
```

```
Orch->>Orch: _set_calibrating(fid, False)
SPA->>Op: complete
```

1. Warmup

- **Purpose** — cycle the fixture through full pan/tilt range so motor belts are thermally and mechanically settled before measurements start. Reduces backlash artifacts in early samples.
- **Preconditions** — valid DMX profile on fixture; Art-Net engine running; calibration lock engaged.
- **Expected duration** — 30 s by default (`warmupSeconds` parameter). Six sub-sweeps (pan±, tilt±, two diagonals) × 20 steps each, ~0.25 s per step.
- **Operator expectation** — beam sweeps visibly across the stage; progress bar creeps from 2% to 8%.
- **Fallback** — if warmup raises an exception, log warning and skip; calibration continues.
- **Cancel** — `_check_cancel()` inside each `_hold_dmx` loop raises `CalibrationAborted`.

2. Discovery

Two code paths exist. The battleship path is preferred when camera homography is reliable; legacy path is the fallback when homography is unavailable (e.g. no surveyed markers visible).

Battleship (preferred, `battleship` + `confirming` status):

- Coarse 4×4 grid at pan/tilt bin centres `{0.125, 0.375, 0.625, 0.875}`² — 16 probes. Per #661, grid density scales with pan range and expected beam width; defaults to 4×4 but can reduce to 3×3 or expand to 5×5 for wide-pan fixtures.
- On the first detected pixel candidate, run the **blink-confirm** routine (#658): nudge pan by `confirm_nudge_delta` (≈ 0.02 of full range) and verify the detected pixel moves; nudge tilt likewise. If **both** pixel deltas exceed `min_delta`, the candidate is confirmed. Otherwise it was a reflection and is rejected — discovery resumes.
- **Expected duration** 10–15 s (16 probes × 0.6 s settle, plus 4 nudges × 0.6 s when a candidate hits).
- **Fallback on failure** — fall through to the legacy coarse+spiral path, or abort to `error`.

Legacy (`discovery` status):

- Initial probe at the warmstart aim (model prediction or geometric estimate from camera FOV).
- Coarse 10×7 grid: pan bins $0.02 + 0.96 \cdot i/9$, tilt bins $0.1 + 0.85 \cdot j/6$ — 70 probes. Each probe goes through `_wait_settled`, which reuses the same adaptive-settle machinery (`SETTLE_BASE = 0.4 s` with `[0.4, 0.8, 1.5] s` escalation) used in the mapping phase — not a separate fixed `SETTLE` constant.
- If the coarse sweep misses, spiral outward from the warmstart aim in rectangular shells at `STEP = 0.05`, up to `MAX_PROBES = 80` total.
- **Expected duration** — 45–55 s worst-case.
- **Fallback on failure** — abort with `error`; call `_cal_blackout()`.

Operator expectations — beam sweeps through a visible grid of positions. If the beam is clearly landing where the camera can see it but detection fails, check §A.5 dark-reference and §A.8 color filter config.

3. Mapping (legacy BFS) / Convergence (v2)

Legacy BFS (`mapping` status):

- BFS from the discovered `(pan, tilt, pixel)` seed. Each step detects the beam and, on success, enqueues four neighbours (up/down/left/right by `STEP`). Beam loss marks the current cell as a visible-region boundary; stale detections (where the pixel barely moves despite a large pan/tilt delta) are rejected as noise.
- Adaptive settle (#655) scales per-probe settle time by movement distance, with escalation levels `[0.4, 0.8, 1.5] s`. Dual-capture with a 0.2 s verify gap and a 30-pixel drift threshold filters mid-move frames; median filtering across the capture pair rejects outliers.
- **Targets** — `_map_target = 50` samples (hard minimum 6; bounded by `MAX_SAMPLES = 80`).
- **Expected duration** — 35–50 s.

v2 Convergence (`sampling` status):

- For each target from `pick_calibration_targets` (filtered through the camera floor-view polygon per #659), converge the beam on the target pixel via `converge_on_target_pixel`.
- **Bracket-and-retry refine (#660)** — initial `bracket_step = 0.08`. When the beam is lost, halve the step and walk back toward the best-known-good offset in the error direction. `BRACKET_FLOOR` is fixture-dependent: $1 / (2^{\text{pan_bits}} - 1)$ — about `0.0039` on 8-bit pan and `0.0000153` on 16-bit, so the loop exhausts to the fixture's actual DMX resolution instead of the old 8-bit-only `1/255` floor (#679). Reset

`bracket_step` to 0.08 on beam re-acquisition. Typical convergence: 5–10 iterations; max 25.

- **Expected duration** — 30–60 s (N targets × ~1 s each).

Fallback — if fewer than 6 samples are collected, abort with `error`.

4. Grid build

- Pure compute: extract unique pan/tilt values from samples, sort, nearest-neighbour fill for missing cells.
- **Expected duration** — <100 ms, no I/O.
- **Fallback** — if sample spread insufficient to form a grid, abort with `error`.

5. Verification sweep

- Pick 3 random targets inside the grid bounds, avoiding fit samples by ≥0.05 pan/tilt margin, with a 10% interior shrink to dodge weak-interpolation edges.
- For each: predict pixel via grid lookup, detect actual beam, compute pixel error.
- **Expected duration** — 3–5 s (3 × ~1 s settle+detect).
- **Advisory only** — logs a warning if any point fails; does **not** block the save.

6. Model fit (parametric, Levenberg-Marquardt)

- **Sign verification (#652 / §8.1) runs FIRST, before the LM fit.** Right after discovery returns `(pan, tilt, pixel)`, the thread issues two extra probes (`pan + 0.02`, `tilt + 0.02`) and calls `verify_signs` on the pixel deltas to compute `(pan_sign, tilt_sign)`. Those signs flow into `fit_model(..., force_signs=(ps, ts))` so the LM runs **one** solve instead of four.
- Fallback — if any sign probe fails to detect a beam, `force_signs` is left `None` and `fit_model` runs the full four-combo search and picks the lowest-RMS candidate.
- When the top two candidates agree to within 0.2° RMS AND the caller didn't supply `force_signs`, `FitQuality.mirror_ambiguity` is set True and surfaced on the status endpoint (#679) so the UI can flag the calibration for a manual re-run.
- `scipy.optimize.least_squares` with `soft_l1` loss (`f_scale=0.05`) fits five continuous parameters (mount yaw/pitch/roll + pan/tilt offsets); up to 120 iterations.
- **Expected duration** — <1 s total (verify_signs + single LM solve).
- **Fallback** — if all 4 combos fail to converge, raise `RuntimeError`; caller aborts with `error`.

7. Held-out parametric gate (#654)

- After fit, drive the fixture to 2–3 **unseen** targets (not used in discovery, mapping, or verification) and measure pixel-level residual against model prediction.
- If residual is within tolerance, set `fixture["moverCalibrated"] = True`.
- If residual exceeds tolerance, return the result to the SPA as an Accept/Retry prompt: operator may accept (flag still set, marked as degraded) or retry calibration from discovery.
- **Expected duration** — 2–5 s.

8. Save + release

- Persist `samples`, `model` dict, `fitQuality` metrics, and per-phase metadata to `desktop/shared/data/fixtures.json`.
- Set `fixture["moverCalibrated"] = True` (if not already).
- Release the calibration lock via `_set_calibrating(fid, False)` — the mover-follow engine resumes writing pan/tilt.
- Blackout the fixture.

B.4 Time budget + blackout-on-timeout (#653)

Each phase has a per-phase wall-clock budget. If exceeded, the phase raises `PhaseTimeout`, caught by the top-level calibration thread, which then:

1. Calls `_cal_blackout()` — sends 512 zeros to the fixture's universe for 0.3 s.
2. Releases the calibration lock.
3. Sets `job["status"] = "error"`, `job["phase"] = "<phase>_timeout"`.
4. Flags `tier-2 handoff` in the status dict so the SPA can suggest the next diagnostic tier.

Default budgets (can be overridden per fixture via `calibrationBudgets` in settings):

PHASE	DEFAULT BUDGET
Warmup	60 s
Discovery	120 s
Mapping / Convergence	180 s
Grid build	10 s
Verification sweep	30 s
Model fit	15 s
Held-out gate	30 s

Total default budget: ~7.5 min, well above the typical 2–4 min runtime.

B.5 Abort path

Cancellation can originate from three sources: operator (`POST /api/calibration/mover/<fid>/cancel`), phase timeout (§B.4), or an unhandled exception. All three converge on the same cleanup:

1. **Foreground immediate blackout** (operator-initiated only): on the `/cancel` request, the orchestrator zeroes the fixture’s channel window on the running Art-Net engine buffer in the foreground, so the next 25 ms frame carries zeros to the bridge — the operator sees the light go off immediately.
2. **Background unwind**: the calibration thread sets `_cancel_event`, which `_check_cancel()` inside `_hold_dmx` picks up and raises `CalibrationAborted`. The exception propagates to `_mover_cal_thread`, which catches it, calls `_cal_blackout()` (512 zeros + release), sets `status = "cancelled"`, `phase = "cancelled"`.
3. **Lock release** — `_set_calibrating(fid, False)` is always called in the cleanup, regardless of which path triggered the cancel.

B.6 Failure modes & operator diagnostics

SYMPTOM	PROBABLE CAUSE	WHAT TO CHECK / TRY
Discovery completes 80 probes without finding beam	Beam too dim, camera can't see it, wrong colour, mover not actually on	Verify fixture is powered and responding; increase threshold; run Fixture Orientation Test (§14); check §A.8 colour configuration; dim room lights
Discovery finds beam immediately but blink-confirm always rejects	Reflective surface (mirror, glass, polished floor) being detected instead of beam	Add diffuse material over the reflector; pick a different beam colour; move the mover's warmstart aim away from the reflector
Mapping / convergence aborts with "fewer than 6 samples"	BFS boundary is too narrow — camera sees only a small slice of the pan/tilt range	Reposition camera to see more of the floor; increase camera count; verify mover's position in layout matches reality
<code>moverCalibrated</code> flag never sets	Held-out gate is failing	Check the Accept/Retry prompt; if residual is reported, an Accept keeps the flag but marks it degraded; a full Retry restarts from discovery
Sign verification logs mirror ambiguity	Fit sees two equally good sign combos	Check physical mover pan/tilt direction against §14 Fixture Orientation Test; may need to toggle Invert Pan/Tilt or Swap Pan/Tilt flags
Calibration "hangs" at a phase	Phase budget (#653) not yet triggered; or Art-Net engine stopped mid-run	Wait up to the phase budget; check engine is running (<code>POST /api/dmx/start</code> if not); if still stuck, cancel and check orchestrator log
Light flashes momentarily then stops, status stays <code>running</code>	Foreground cancel happened but background thread is still unwinding	Normal; background cleanup completes within 1-2 s

B.7 Tuning-parameter reference

Operator-tunable (#680) — Settings → Advanced → Calibration Timeouts. The constants below are the shipped defaults; operators can override them via the Advanced panel, which persists into `desktop/shared/data/settings.json` under `calibrationTuning`. Overrides are read at phase start, so a change takes effect on the next calibration run without a server restart. Validation is centralised in `CAL_TUNING_SPEC` (`parent_server.py`); `POST /api/settings` rejects out-of-range values with a 400. Every constant below has a

matching entry in `CAL_TUNING_SPEC` except the ones flagged “fixed” (e.g. `STEP`, `MAX_SAMPLES` is exposed as `bfsMaxSamples`, `BRACKET_FLOOR` is derived per fixture and not operator-tunable per #679).

Constants in `desktop/shared/mover_calibrator.py`:

CONSTANT	DEFAULT	ROLE
<code>SETTLE</code> (legacy)	0.6 s	Per-probe settle before detection
<code>SETTLE_BASE</code> (#655)	0.4 s	Adaptive-settle base before escalation
<code>SETTLE_ESCALATION</code>	<code>[0.4, 0.8, 1.5]</code> s	Escalation tiers on pixel-drift retry
<code>SETTLE_VERIFY_GAP</code>	0.2 s	Dual-capture spacing for median filter
<code>SETTLE_PIXEL_THRESH</code>	30 px	Inter-capture drift threshold for “settled”
<code>STEP</code>	0.05	Fine spiral step size (normalized pan/tilt)
<code>MAX_SAMPLES</code>	80	Hard cap on BFS samples
<code>COARSE_PAN</code>	10	Legacy coarse grid pan bins
<code>COARSE_TILT</code>	7	Legacy coarse grid tilt bins
<code>BRACKET_FLOOR</code>	$1 / (2^{\text{pan_bits}} - 1)$	Convergence refine floor — fixture-dependent (8-bit → <code>≈0.0039</code> , 16-bit → <code>≈0.0000153</code>) (#679)

Constants in `desktop/shared/mover_control.py`:

CONSTANT	DEFAULT	ROLE
Claim TTL (<code>moverClaimTtlS</code> , #680)	15 s	Auto-release if claim not refreshed

Phase time budgets (from `CAL_TUNING_SPEC` in `desktop/shared/parent_server.py`, #680):

KEY	DEFAULT	CLAMP	ROLE
<code>discoveryBattleshipS</code>	60 s	20 – 300	Coarse-grid scan budget
<code>discoveryColourFallbackS</code>	90 s	30 – 300	Legacy colour-filter fallback budget
<code>mappingS</code>	120 s	30 – 600	BFS mapping budget (soft cap)
<code>fitS</code>	10 s	5 – 60	LM + grid build
<code>verificationS</code>	30 s	5 – 120	Grid + parametric held-out verify
<code>warmupSeconds</code>	30 s	0 – 120	Pre-cal motor warmup sweep
<code>bfsMaxSamples</code>	80	20 – 300	Hard cap on BFS samples
<code>convergeMaxIterations</code>	25	5 – 100	Per-target bracket-and-retry loop
<code>battleshipPan/TiltStepsMin/Max</code>	3 / 8 / 3 / 6	see spec	Adaptive-grid clamps (#661)

B.8 Related features

- **Unified mover control** — see CLAUDE.md §“Unified mover control (gyro + Android)”. Calibration must complete and `moverCalibrated` must be set before manual mover-control (gyro/phone) will use the parametric model; without it, the control layer falls back to raw DMX pan/tilt passthrough.
 - **Fixture Orientation Test** — §14 “Fixture Orientation Test”. Run this first if the pan/tilt axes in the physical fixture don’t match expectations.
 - **Track Action** — §8 “Track Action”. Consumes the interpolation grid + parametric model to aim at moving subjects.
-

Appendix C — Documentation Maintenance

This appendix describes the contract between the calibration appendices above and the source code that implements them. It exists for issue [#662](#) and is kept short — full details are in `docs/DOCS_MAINTENANCE.md`.

C.1 Source-of-truth files

Any PR that changes calibration behaviour in one of these files is expected to include an Appendix A or B review in the same PR:

Mover calibration: `desktop/shared/mover_calibrator.py`, `mover_control.py`, `parametric_mover.py`, `desktop/shared/spa/js/calibration.js`, `desktop/shared/parent_server.py` (routes `/api/calibration/mover/*`).

Camera calibration: `firmware/orangepi/camera_server.py`, `beam_detector.py`, `depth_estimator.py`, `desktop/shared/space_mapper.py`, `surface_analyzer.py`, `camera_math.py`, `desktop/shared/parent_server.py` (routes `/api/aruco/markers*`, `/api/cameras/<fid>/stage-map`, `/api/cameras/<fid>/aruco/*`, `/api/cameras/<fid>/intrinsic*`, `/api/cameras/<fid>/beam-detect`, `/api/space/scan`).

C.2 Reviewer checklist (short form)

On a calibration-touching PR, confirm:

- Phase names in `mover_calibrator.py` match the Appendix B §B.2 table
- Timeout constants in the §B.7 table still match code
- Endpoint paths + request/response shapes in Appendix A match Flask route signatures
- Rotation-schema v2 (§A.9) still matches `camera_math.py::rotation_from_layout`
- Status strings written to the calibration-status dict match the state machine diagram

The full checklist, including render verification for the Mermaid diagrams under `docs/diagrams/` and the DRAFT-banner removal criteria, is in `docs/DOCS_MAINTENANCE.md`.

C.3 Regenerating the manual

- Canonical source: `docs/USER_MANUAL.md` (this file).
- `docs/SlyLED_User_Manual.docx` + `.pdf` are **built separately** by `tests/build_manual.py`, which constructs the document from scratch rather than

parsing this markdown. The docx/PDF path does not yet include these appendices — follow-up work.

- Diagram sources live in `docs/diagrams/*.mmd`. Mermaid blocks are embedded inline in the markdown so GitHub renders them directly; external renderers like Kroki can generate SVG/PNG from the standalone files for PDF inclusion.

C.4 Enforcement

No automatic drift-check is wired up yet. Proposed options, in order of cost:

1. PR-template checkbox (`.github/pull_request_template.md`)
2. GitHub Actions grep: fail PRs that touch the source-of-truth list without touching `docs/USER_MANUAL.md`, with a skip-override label
3. Scheduled drift agent (weekly)

These require `.github/` changes and are tracked as follow-ups under #662.

C.5 DRAFT banner removal

The DRAFT banners on Appendix A and B should be removed once the in-flight items listed in `docs/DOCS_MAINTENANCE.md §"When to bump the DRAFT banner"` are all confirmed merged. At the time this appendix was drafted (2026-04-23), the following are known to be partial or not yet in code: #653 time budgets, #654 held-out parametric gate, #655 full median oversample, #658 blink-confirm on non-battleship path, #659 floor-view polygon target filter, #661 adaptive battleship density.

Appendix D — Idle Behaviour

A “rock-solid” rig is one where, at any moment a show isn’t playing and an operator isn’t actively driving a head, every moving head parks at a known pose with the lamp closed. Operators expect that contract because the alternative — a head sitting on whatever pose the last writer happened to leave it in — looks broken even when nothing is wrong. This appendix is the canonical list of when the orchestrator parks, when it doesn’t, and what “parked” means in practice.

What “parked” means

A parked moving head:

- Aims at its **Home** pose (the pose saved in the Set Home wizard; fallback to mechanical centre when no Home is saved).
- Holds **dimmer = 0** so the lamp doesn't bleed onto the rig.
- Closes the **shutter** if the profile carries a strobe channel with a **Closed** capability — fixtures with mechanical shutters benefit from the explicit close.
- Releases any colour-wheel slot it was holding back to slot 0 (open / white) so the next show frame doesn't inherit a stale filter.

When the orchestrator parks a head

TRIGGER	PATH	NOTES
Cold start	Orchestrator boot	Every DMX fixture parks once the engine comes up. Avoids the “head was left aimed at the back wall last night” surprise.
Timeline natural end	<code>_dmx_playback_loop</code> exit	Heads driven by the timeline’s bake park when the show finishes. Track-action-driven heads also park (#807) — prefix only the bake-driven heads parked, leaving any tracker-claimed mover stuck at its last pose.
Operator presses Stop	<code>_dmx_playback_stop</code> set	Same as natural end. The blackout sweep applies only on stop or final-iteration end (#840), not between loop iterations.
Claim release	Mover-control claim arbiter	When an Android phone or gyro puck releases a claim, the head returns to the show if a show is running, otherwise parks. The release is instant — no slewed easing in v1.7.83+.
Power-cycle re-settle	First PONG from a child after a boot	When a fixture’s child board power-cycles, the orchestrator resends the current globalBrightness (#843) and the next show frame writes a known pose. Pre-v1.7.83 the child could come up at full brightness for one frame; the PONG-time top-up closes that window.

What does NOT trigger a park

These intentionally do not park heads — operators sometimes expect them to, but parking on every one-shot would steal the head from the show.

- **One-shot** `/api/mover/<fid>/aim` — these are operator-direct test pulses; the rig holds whatever pose the route sets until the next show frame.
- **DMX-test sliders on the Settings tab** — same reasoning. The sliders override show output for as long as the operator is driving them.
- **Brief gaps within an active claim** — a phone or puck temporarily losing WiFi for a second doesn't release the claim. The remote-control claim TTL is 15 s; a head only parks when the TTL elapses without a heartbeat (§6.3 “all-comms silence”).
- **Calibration probes** — beam discovery and convergence sweeps hold the head wherever the probe lands. The calibration session parks the head explicitly when it finishes (success or abort).

How to verify on your rig

1. Park a moving head at a known wall (Set Home → save).
2. Stop any running show.
3. Aim the head at the floor with `/api/mover/<fid>/aim`.
4. Press play on a 5-second blank timeline. The head should NOT move (rule three: one-shot aims hold).
5. Stop the timeline. The head should return to Home with the lamp closed within ~50 ms.

If step 5 doesn't happen, check (a) the fixture has a Home pose saved, (b) the timeline ended naturally rather than crashing, (c) no claim is held on the fixture (the Setup tab shows claim state per fixture).

Appendix E — Remote Control: Android Phone & Gyro Puck

Two remote controllers can drive moving heads in real time alongside a running show: an Android phone running the SlyLED operator app and a WaveShare ESP32-S3 round-LCD gyro puck. Both go through the same claim arbiter on the orchestrator, both follow the same handshake protocol, and both cooperate with the show timeline through the mover-control claim arbiter. This appendix describes the full lifecycle, the gestures, and how claim arbitration interacts with preset shows.

Claim lifecycle

The claim is the orchestrator’s “this remote currently owns mover N” lock. It carries a 16-bit nonce, a TTL, and a current pose so the system can reconcile the puck’s UI state with the orchestrator’s arbiter state when one of them reboots or drops a packet.

1. IDLE on remote.
2. Operator presses Start (puck) or Claim (Android).
3. Remote ships CMD_GYRO_START / claim request with a fresh 16-bit nonce.
4. Orchestrator allocates a mover, replies CLAIM_ACK with the nonce + assigned moverId. The remote advances UI to ACTIVE only on a matching ACK; CLAIM_DENIED reverts; ~1.5 s overall timeout reverts with "NO RESPONSE".
5. Remote streams orient quaternions at ~50 Hz; orchestrator converts to aim-stage and writes pan / tilt to the head.
6. Both ends exchange 2 s heartbeats (HB_REP carries uiState + claimNonce + seq) so divergent state is reconciled.
7. Operator presses Stop / Release; remote ships nonce; orchestrator replies STOP_ACK and releases the claim.

The full state-machine spec lives in `docs/gyro-claim-lifecycle.md` and is the source of truth for any change to the protocol.

What the operator sees

- **Press Start on the puck** — page advances to “ACTIVE” within ~150 ms. If the orchestrator can’t claim a mover (none online, none available), the page reverts to IDLE with a denial reason.
- **Press Stop on the puck** — page returns to IDLE; the head returns to whatever the show was driving (or parks if no show is running).
- **Calibrate** — hold the **Calibrate** button (puck or Android) for as long as you need; release to capture the new reference pose. The screen advances to the colour picker page on the puck; the Android app advances to the gesture page.
- **Connection lost** — both remotes show a stale-reason badge if the orchestrator stops hearing heartbeats. The puck self-clears when it resumes streaming (#812 / #821 / #823); operator can also force-clear via `POST /api/remotes/<id>/clear-stale`.

Gestures

Once active, both remotes drive the same `aim_stage` semantic — the head’s beam aims at a stage-coordinate point computed from the remote’s orientation.

Phone (Android)

- **Pitch** (tip phone forward / back) — beam pitches up / down on the head.
- **Roll** (tilt phone left / right) — beam pans across the stage.
- **Yaw** (rotate phone around vertical) — beam pans across the stage.
- **Volume buttons** — fine dimmer up / down (Android operator app configurable).
- **Auto Brightness** (chapter on Brightness) — the app can drive the orchestrator's master brightness from the local mic envelope at ~20 Hz, gamma-scaled to the rig (#820, #843).

The phone-specific yaw axis is mirrored relative to the puck (#824) because the phone's natural-portrait orientation puts the operator's "left" 90 ° offset from the puck's body frame. The operator never needs to think about this; the orchestrator's `_apply_quat` for `KIND_PHONE` handles the negation.

Gyro puck

- **Pitch** (tip puck forward / back) — beam pitches up / down.
- **Yaw** (rotate around the puck's vertical axis) — beam pans.
- **Roll** (tilt left / right) — colour-wheel selection on profiles with a colour wheel; ignored on RGB-only profiles.
- **Press Start** — claim mover and start streaming.
- **Press Stop** — release claim.
- **Press Calibrate** — capture new reference pose.

Claim arbitration with shows (#763)

Claims take priority over the show timeline:

- A claimed head is **muted** from `_evaluate_track_actions` and from the bake-driven `set_fixture_dimmer` / `set_fixture_pan_tilt` writes. The claim writer owns the head until release.
- Other heads in the rig keep playing the show normally — the claim affects only the assigned mover.
- On release the head **rejoins the show within one frame**: no slew, no fade. If the show has moved on, the head jumps to wherever the show currently has it. This was a deliberate choice per #763 — slewing back in took the operator out of the moment; snap-rejoin is what real consoles do.

- Track actions evaluate every frame, so a head that was claimed during a sweep returns to wherever the sweep is **right now**, not to where the sweep would have been mid-claim.

Colour & dimmer during a claim (#814)

A claim doesn't take over colour or dimmer:

- The remote's gestures drive **only pan / tilt** (and colour wheel for the puck's roll axis on profiles that support it).
- The head's dimmer and RGB stay under the show's control. If the show is dim, the claimed head stays dim — the operator picks pan and tilt; the show paints colour and intensity.
- This applies to global brightness (#843) the same way: a claim during Auto Brightness inherits the auto-driven master.

Recovery from divergent state

The handshake's heartbeats include both ends' state, so divergent combinations are reconciled:

PUCK UI	ORCHESTRATOR	WHAT HAPPENS
ACTIVE	claim held	Normal — heartbeats keep TTL alive.
ACTIVE	no claim	Orchestrator reconstructs the claim (orchestrator-restart bootstrap).
IDLE	claim held	Orphan claim — orchestrator releases it.
IDLE	no claim	Normal idle.

The orphan-claim guard fires 1.5 s after CLAIM_ACK if no orient arrives, releasing the claim so a wedged remote can't squat on a mover indefinitely.
